

LAYER: 2_MesCore

This file is a merged representation of a subset of the codebase, containing files not matching ignore patterns, combined into a single document by Repomix.

<file_summary>

This section contains a summary of this file.

<purpose>

This file contains a packed representation of a subset of the repository's contents that is considered the most important context.

It is designed to be easily consumable by AI systems for analysis, code review, or other automated processes.

</purpose>

<file_format>

The content is organized as follows:

1. This summary section
2. Repository information
3. Directory structure
4. Repository files (if enabled)
5. Multiple file entries, each consisting of:
 - File path as an attribute
 - Full contents of the file

</file_format>

<usage_guidelines>

- This file should be treated as read-only. Any changes should be made to the original repository files, not this packed version.
- When processing this file, use the file path to distinguish between different files in the repository.
- Be aware that this file may contain sensitive information. Handle it with the same level of security as you would the original repository.

</usage_guidelines>

<notes>

- Some files may have been excluded based on .gitignore rules and Repomix's configuration
- Binary files are not included in this packed representation. Please refer to the Repository Structure section for a complete list of file paths, including binary files
- Files matching these patterns are excluded: `**/obj/**`, `**/bin/**`, `**/*.png`, `**/*.jpg`, `**/node_modules/**`, `**/*.Designer.cs`
- Files matching patterns in .gitignore are excluded
- Files matching default ignore patterns are excluded
- Files are sorted by Git change count (files with more changes are at the bottom)

</notes>

</file_summary>

<directory_structure>

Promatis.Net.MES.Configuration/Class1.cs
Promatis.Net.MES.Configuration/Promatis.Net.MES.Configuration.csproj
Promatis.Net.MES.Data/Configuration/TechnologicalOperationParameterBaseConfiguration.cs
Promatis.Net.MES.Data/Configuration/TechnologicalOperationUnitBaseConfiguration.cs
Promatis.Net.MES.Data/Configuration/UnitBaseConfiguration.cs
Promatis.Net.MES.Data/DBContext/MesApplicationDbContext.cs
Promatis.Net.MES.Data/Promatis.Net.MES.Data.csproj
Promatis.Net.MES.Data/Triggers/TechnologicalOperationHierarchyTrigger.cs
Promatis.Net.MES.Data/Triggers/UnitBaseHierarchyTrigger.cs
Promatis.Net.MES.Domain.Interface/Enums/CalculationMethod.cs
Promatis.Net.MES.Domain.Interface/Enums/UnitKind.cs
Promatis.Net.MES.Domain.Interface/Enums/UnitType.cs
Promatis.Net.MES.Domain.Interface/Interfaces/ITechnologicalOperation.cs
Promatis.Net.MES.Domain.Interface/Interfaces/ITechnologicalParameter.cs
Promatis.Net.MES.Domain.Interface/Promatis.Net.MES.Domain.Interface.csproj
Promatis.Net.MES.Domain/Promatis.Net.MES.Domain.csproj
Promatis.Net.MES.Domain/TechnologicalOperation/TechnologicalOperationBase.cs
Promatis.Net.MES.Domain/TechnologicalOperation/TechnologicalOperationBaseValidator.cs
Promatis.Net.MES.Domain/TechnologicalOperation/TechnologicalOperationHierarchyEngine.cs
Promatis.Net.MES.Domain/TechnologicalOperationParameter/TechnologicalOperationParameterBase.cs
Promatis.Net.MES.Domain/TechnologicalOperationParameter/TechnologicalOperationParameterBaseValidator.cs
Promatis.Net.MES.Domain/TechnologicalOperationUnit/TechnologicalOperationUnitBase.cs
Promatis.Net.MES.Domain/TechnologicalOperationUnit/TechnologicalOperationUnitBaseValidator.cs
Promatis.Net.MES.Domain/TechnologicalParameter/TechnologicalParameterBase.cs
Promatis.Net.MES.Domain/TechnologicalParameter/TechnologicalParameterBaseValidator.cs
Promatis.Net.MES.Domain/TechnologicalParameterCalcMethod/TechnologicalParameterCalcMethodBase.cs

```
Promatis.Net.MES.Domain/TechnologicalParameterCalcMethod/
TechnologicalParameterCalcMethodBaseValidator.cs
Promatis.Net.MES.Domain/TechnologicalParameterValue/TechnologicalParameterValueBase.cs
Promatis.Net.MES.Domain/TechnologicalParameterValue/TechnologicalParameterValueBaseValidator.cs
Promatis.Net.MES.Domain/UnitBase/UnitBase.cs
Promatis.Net.MES.Domain/UnitBase/UnitBaseValidator.cs
Promatis.Net.MES.Domain/UnitBase/UnitHierarchyEngine.cs
Promatis.Net.MES.Domain/UnitOfMeasurement/UnitOfMeasurement.cs
Promatis.Net.MES.Domain/UnitOfMeasurement/UnitOfMeasurementValidator.cs
Promatis.Net.MES.Service/Promatis.Net.MES.Service.csproj
Promatis.Net.MES.Service/TechnologicalOperationParameterService/
ITechnologicalOperationParameterService.cs
Promatis.Net.MES.Service/TechnologicalOperationParameterService/
TechnologicalOperationParameterService.cs
Promatis.Net.MES.Service/TechnologicalOperationService/ITechnologicalOperationService.cs
Promatis.Net.MES.Service/TechnologicalOperationService/TechnologicalOperationService.cs
Promatis.Net.MES.Service/TechnologicalParameterCalcMethodService/
ITechnologicalParameterCalcMethodService.cs
Promatis.Net.MES.Service/TechnologicalParameterCalcMethodService/
TechnologicalParameterCalcMethodService.cs
Promatis.Net.MES.Service/TechnologicalParameterService/ITechnologicalParameterService.cs
Promatis.Net.MES.Service/TechnologicalParameterService/TechnologicalParameterService.cs
Promatis.Net.MES.Service/UnitOfMeasurementService/IUnitOfMeasurementService.cs
Promatis.Net.MES.Service/UnitOfMeasurementService/UnitOfMeasurementService.cs
Promatis.Net.MES.Service/UnitService/IUnitBaseService.cs
Promatis.Net.MES.Service/UnitService/UnitBaseService.cs
Promatis.Net.MES.Tests/AssemblyInfo.cs
Promatis.Net.MES.Tests/Promatis.Net.MES.Tests.csproj
Promatis.Net.MES.Tests/Service/UnitServiceStandaloneTests.cs
Promatis.Net.MES.Tests/UnitBase/AppDbContext.cs
Promatis.Net.MES.Tests/UnitBase/TestUnit.cs
Promatis.Net.MES.Tests/UnitBase/TestUnitValidator.cs
Promatis.Net.MES.Tests/UnitBase/UnitBaseArchitectureTests.cs
Promatis.Net.MES.UI/_Imports.razor
Promatis.Net.MES.UI/Pages/UnitOfMeasurements/UnitOfMeasurementDialog.razor
Promatis.Net.MES.UI/Pages/UnitOfMeasurements/UnitOfMeasurementDialog.razor.cs
Promatis.Net.MES.UI/Pages/UnitOfMeasurements/UnitOfMeasurementPage.razor
Promatis.Net.MES.UI/Pages/UnitOfMeasurements/UnitOfMeasurementPage.razor.cs
Promatis.Net.MES.UI/Pages/UnitOfMeasurements/UnitOfMeasurementWorkspaceContext.cs
Promatis.Net.MES.UI/Promatis.Net.MES.UI.csproj
Promatis.Net.MES.UI/UiModule.cs
Promatis.Net.MES.UI/wwwroot/exampleJsInterop.js
</directory_structure>
```

<files>

This section contains the contents of the repository's files.

<file path="Promatis.Net.MES.Configuration/Class1.cs">

namespace Promatis.Net.MES.Configuration

{

public class Class1

{

}

}

</file>

<file path="Promatis.Net.MES.Configuration/Promatis.Net.MES.Configuration.csproj">

<Project Sdk="Microsoft.NET.Sdk">

<PropertyGroup>

<TargetFramework>net10.0</TargetFramework>

<ImplicitUsings>enable</ImplicitUsings>

<Nullable>enable</Nullable>

</PropertyGroup>

<ItemGroup>

<ProjectReference Include="..\..\Core\Configuration.Web\Promatis.Net.Configuration.Web.csproj" /

>

<ProjectReference Include="..\Promatis.Net.MES.Service\Promatis.Net.MES.Service.csproj" />

</ItemGroup>

</Project>

</file>

<file path="Promatis.Net.MES.Data/Configuration/
TechnologicalOperationParameterBaseConfiguration.cs">

```

using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using Promatis.Net.Domain;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;

namespace Promatis.Net.MES.Data;

public abstract class TechnologicalOperationParameterBaseConfiguration<T, TOperation, TParameter> :
    IEntityTypeConfiguration<T>
{
    where T : TechnologicalOperationParameterBase<TOperation, TParameter> // <--- A05D 5CD0CT< C0;CâAC08CR ?C
    where TOperation : DomainObject, ITechnologicalOperation
    where TParameter : DomainObject, ITechnologicalParameter
{
    public virtual void Configure(EntityTypeBuilder<T> builder)
    {
        // 1. B4=C,,:C ;DÄ=D'9 C,,=CD5C0A CD;D0 ?D 5CD>D$2D 0D"5C08D0 4D41C'8D >C$0C08D0 ?C @C <CTBD >C" =C >C
        builder.HasIndex(x => new { x.OperationId, x.ParameterId })
            .IsUnique();
    }
}
}
</file>

```

```

<file path="Promatis.Net.MES.Data/Configuration/TechnologicalOperationUnitBaseConfiguration.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using Promatis.Net.Domain;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;

namespace Promatis.Net.MES.Data;

/// <summary>
/// A0>C0DC,,3D4@C FC,,O CD;D0 1C 7D2 AC$0Ct8D
/// </summary>
/// <typeparam name="T"></typeparam>
public abstract class TechnologicalOperationUnitBaseConfiguration<T, TOperation> :
    IEntityTypeConfiguration<T>
{
    where T : TechnologicalOperationUnitBase<TOperation> // <--- A05D 5CD0CT< D$8C0 >C05D 0Dd8C•
    where TOperation : DomainObject, ITechnologicalOperation
{
    public virtual void Configure(EntityTypeBuilder<T> builder)
    {
        // 1. B4=C,,:C ;DÄ=D'9 C,,=CD5C0A CD;D0 ?C @D2 ?Cä;CT9 (A0@CT4CâBC$@C IC 5D" 4D41C'8D >C$0C08CR AC$0Ct5
        builder.HasIndex(x => new { x.OperationId, x.UnitId })
            .IsUnique();
    }
}
}
</file>

```

```

<file path="Promatis.Net.MES.Data/Configuration/UnitBaseConfiguration.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using Promatis.Net.MES.Domain;

namespace Promatis.Net.MES.Data;

public class UnitBaseConfiguration : IEntityTypeConfiguration<UnitBase>
{
    public void Configure(EntityTypeBuilder<UnitBase> builder)
    {

```

```

        // A$:C'NDt0CT< D BD 0D$5C48Dâ E B 4C'O C$ACT9 C,,5D 0D EC,,8 UnitBaseD
        builder.UseTptMappingStrategy();D
D
        // A00D BD >C":C ?Cä;CT9D
        builder.Property(x => x.Kind).IsRequired();D
        builder.Property(x => x.Type).IsRequired();D
    }D
}
</file>

<file path="Promatis.Net.MES.Data/DbContext/MesApplicationDbContext.cs">
using Microsoft.EntityFrameworkCore;D
using Microsoft.Extensions.Configuration;D
using Promatis.Net.Data;D
using Promatis.Net.MES.Domain;D
D
namespace Promatis.Net.MES.Data;D
D
public class MesApplicationDbContext(D
    DbContextOptions options,D
    IConfiguration configuration,D
    IServiceProvider? serviceProvider = null) D
    : ApplicationDbContext(options, configuration, serviceProvider)D
{D
    public DbSet<UnitOfMeasurement> UnitOfMeasurements => Set<UnitOfMeasurement>();D
}
</file>

<file path="Promatis.Net.MES.Data/Promatis.Net.MES.Data.csproj">
<Project Sdk="Microsoft.NET.Sdk">D
D
    <PropertyGroup>D
        <TargetFramework>net10.0</TargetFramework>D
        <ImplicitUsings>enable</ImplicitUsings>D
        <Nullable>enable</Nullable>D
    </PropertyGroup>D
D
    <ItemGroup>D
        <PackageReference Include="Microsoft.EntityFrameworkCore" Version="10.0.8" />D
        <PackageReference Include="Microsoft.EntityFrameworkCore.Relational" Version="10.0.8" />D
        <PackageReference Include="Microsoft.Extensions.Configuration" Version="10.0.8" />D
    </ItemGroup>D
D
    <ItemGroup>D
        <ProjectReference Include="..\..\Core\Data\Promatis.Net.Data.csproj" />D
        <ProjectReference Include="..\Promatis.Net.MES.Domain\Promatis.Net.MES.Domain.csproj" />D
    </ItemGroup>D
D
</Project>
</file>

<file path="Promatis.Net.MES.Data/Triggers/TechnologicalOperationHierarchyTrigger.cs">
using Microsoft.EntityFrameworkCore;D
using Promatis.Net.Data;D
using Promatis.Net.MES.Domain;D
D
namespace Promatis.Net.MES.Data;D
D
/// <summary>D
/// A 1D BD 0C=BC0KC' 1C 7Cä2D'9 D$@C,,3C45D 8CT@C @DT8C, BCTEC0>C'>C48Dt5D :C,,E Cä?CT@C FC,,9.D
/// At0D"8D"0CTB D BD CC=BD4@D2 >D" =CT:Cä@D 5C=BC0>C4> CD>C 0C$;CT=C,,O C0>CDCct;Cä2 C, :Cä=D$@Cä;C,,@D45D" AD
/// </summary>D
public abstract class TechnologicalOperationHierarchyTrigger<T, TOLink, TPLink> :
IBeforeSaveTrigger<T>D
    where T : TechnologicalOperationBase<T, TOLink, TPLink>D
    where TOLink : classD
    where TPLink : classD
{D
    public async Task HandleAsync(EntityCancelEventArgs<T> args)D
    {D
        T operation = args.Entity;D
D
        // ATAC'8 Cä?CT@C FC,,O C= >D =CT2C 0 – C0@Cä?D4AC=0CT< C0@Cä2CT@C=C0D
        if (!operation.ParentId.HasValue || operation.ParentId == Guid.Empty) return;D
D
        // A,,7C$;CT:C 5CÄ 8Cr !B4 AB AD$0D$CD –4ÆV b 8 Name D >CD8D$5C'LD :Cä9 Cä?CT@C FC,,8D

```

```

var parentInfo = await args.Context.Set<T>()
    .Where(x => x.Id == operation.ParentId)
    .Select(x => new { x.IsLeaf, x.Name })
    .FirstOrDefaultAsync();

if (parentInfo != null)
{
    // A,"Aô>C`LCtCCT< C0D, 4Cä<CT=C0KC' 4C$8Cd>C 4C`O C0@Cä2CT@C=8 C0@C 2C,,; C$;Cä6CT=C0>D BC,i
    // A05D 5CD0CT< D BC BD4A IsLeaf D >CD8D$5C`O, C$KD$0D"5C0=D`9 C,,7 C 0CtK.D
    if (parentInfo.IsLeaf)
    {
        args.Cancel = true;
        args.ErrorMessage = $"A00D CD,,5C08CR 8CT@C @DT8C, 0U3  "CTEC0>C`>C48Dt5D :C O Cä?CT@C FC,,O '{
            $"C05 CÄ>Cd5D" 1D`BDÄ 2C`>Cd5C00 C" w. &VçD-æf0ää ÖW0rÄ BC : C=0C  >C00
        };
        return;
    }
}

// AD>C0>C`=C,,BCT;DÄ=C O Ct0D"8D$0 C00 C,,7CÄ5C05C08CS 5D ;C, >C05D 0Dd8D0 AD$0C0>C$8D$AD0 C,,AD$>CÄÄ
// C0@Cä2CT@D05CÄ 2 ChangeTracker C,,;C, ABÄ =CTB C`8 D2 =CT5 CD>Dt5D =C,,E D0;CT<CT=D$>C-
if (operation.IsLeaf)
{
    bool hasChildrenInDb = await args.Context.Set<T>()
        .AnyAsync(x => x.ParentId == operation.Id);

    if (!TechnologicalOperationHierarchyEngine.CanChangeLeafStatus(hasChildrenInDb,
        operation.IsLeaf))
    {
        args.Cancel = true;
        args.ErrorMessage = $"A00D CD,,5C08CR FCT;CäAD$=CäAD$8 CD0C0=D`E: A05C$>Ct<Cä6C0> D 4CT;C BDÄ
            $"D$0C  :C : C$=D4BD 8 C05CR CCd5 D >CD5D 6C BD O CD@D43C,,5 D$5DT=Cä;Cä3C
        ";
    }
}
}
</file>

```

```

<file path="Promatis.Net.MES.Data/Triggers/UnitBaseHierarchyTrigger.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Data;
using Promatis.Net.MES.Domain;
namespace Promatis.Net.MES.Data;

/// <summary>
/// B$@C,,3C45D 4C`O C0@Cä2CT@C=8 D ?CTFC,,DC,,GC0KDR ?D 0C$8C^ 8CT@C @DT8C, Væ-D& 6Ri
/// </summary>
public class UnitBaseHierarchyTrigger : IBeforeSaveTrigger<UnitBase>
{
    public async Task HandleAsync(EntityCancelEventArgs<UnitBase> args)
    {
        UnitBase unit = args.Entity;

        if (!unit.ParentId.HasValue || unit.ParentId == Guid.Empty) return;

        // A0>C`CDt0CT< Kind C, æ ÖR @Cä4C,,BCT;D0 4C`O C,,=DD>D <C BC,,2C0>C4> D >Cä1D"5C08Dý
        var parentInfo = await args.Context.Set<UnitBase>()
            .Where(x => x.Id == unit.ParentId)
            .Select(x => new { x.Kind, x.Name })
            .FirstOrDefaultAsync();

        if (parentInfo != null && !UnitHierarchyEngine.IsHierarchyValid(parentInfo.Kind, unit.Kind))
        {
            args.Cancel = true;
            args.ErrorMessage = $"A00D CD,,5C08CR 8CT@C @DT8C, 0U3  C JCT:D" w.Væ-Bäæ ÖW0r ‡.Væ-Bä¶-æG0' " ½
                $"C05 CÄ>Cd5D" 1D`BDÄ 2C`>Cd5C0 2 '{parentInfo.Name}' ({parentInfo.Kind}).";
        }
    }
}
</file>

```

```

<file path="Promatis.Net.MES.Domain.Interface/Enums/CalculationMethod.cs">
using System.ComponentModel;
namespace Promatis.Net.MES.Domain.Interface;

```

```

/// <summary>Ð
/// AÄ5D$>CDK D 0D GCTBC BCTE. C00D 0CÄ5D$@Cä2Ð
/// </summary>Ð
[Flags]Ð
public enum CalculationMethodÐ
{Ð
    [Description("A05 Ct0CD0C0"•Ý
    None = 0, // 00000000Ð
Ð
    [Description("AÄ0C¤AC,,<C ;DÄ=Cä5 Ct=C GCT=C,,5 (MAX)")]Ð
    Max = 1,Ð
Ð
    [Description("AÄ8C08CÄ0C´LC0>CR 7C00Dt5C08CR „0"á'"•Ý
    Min = 2,Ð
Ð
    [Description("B @CT4C05CR 7C00Dt5C08CR „ dr'"•Ý
    Avg = 4,Ð
Ð
    [Description("A05D 2Cä5 Ct=C GCT=C,,5 (FIRST)")]Ð
    First = 8,Ð
Ð
    [Description("A0>D ;CT4C05CR 7C00Dt5C08CR „Ä 5B'"•Ý
    Last = 16,Ð
Ð
    [Description("A$ACR 7C00Dt5C08D0 „ ÄÄ'"•Ý
    All = 32Ð
}
</file>

<file path="Promatis.Net.MES.Domain.Interface/Enums/UnitKind.cs">
using System.ComponentModel;Ð
Ð
namespace Promatis.Net.MES.Domain.Interface;Ð
Ð
public enum UnitKindÐ
{Ð
    [Description("B :C´0CDAC¤0D0 ;Cä3C,,AD$8C¤0")]Ð
    Storage = UnitType.Storage | UnitType.Zone | UnitType.Rack | UnitType.Cell | UnitType.Crane,Ð
Ð
    [Description("A0@Cä8Ct2Cä4D BC$5C0=C 0 Ct>C00")]Ð
    Production = UnitType.Workshop | UnitType.Section | UnitType.Line | UnitType.Workstation |
UnitType.MachineTool | UnitType.Table,Ð
Ð
    [Description("B$@C =D ?Cä@D$=D´9 D47CT;")]Ð
    Transport = UnitType.Vehicle | UnitType.Conveyor,Ð
Ð
    [Description("A0>CD@C 7CD5C´5C08CR"•Ý
    Department = UnitType.Workshop | UnitType.Section | UnitType.Other,Ð
Ð
    [Description("B 0C >Dt0D0 BCäGC¤0 / B0GCT9C¤0")]Ð
    Position = UnitType.Cell | UnitType.OtherÐ
}
</file>

<file path="Promatis.Net.MES.Domain.Interface/Enums/UnitType.cs">
using System.ComponentModel;Ð
Ð
namespace Promatis.Net.MES.Domain.Interface;Ð
Ð
[Flags]Ð
public enum UnitTypeÐ
{Ð
    None = 0,Ð
Ð
    [Description("Bd5DR"•Ý
    Workshop = 1,Ð
Ð
    [Description("B4GC AD$>C¤"•Ý
    Section = 2,Ð
Ð
    [Description("A´8C08D0 0 Cä=C$5C"5D "•Ý
    Line = 4,Ð
Ð
    [Description("B 0C >Dt5CR <CTAD$>")]Ð
    Workstation = 8,Ð
Ð
}

```

```

        [Description("B :C`0CB"•Ý
Storage = 16,Đ
Đ
        [Description("At>C00 DT@C =CT=C,,0"))]Đ
Zone = 32,Đ
Đ
        [Description("B BCT;C`0Cb"•Ý
Rack = 64,Đ
Đ
        [Description("B0GCT9C00 C 4D 5D 0 DT@C =CT=C,,0"))]Đ
Cell = 128,Đ
Đ
        [Description("A0@C = / A0>CDJCT<C08C0"•Ý
Crane = 256,Đ
Đ
        [Description("B BC =Cä:")]Đ
MachineTool = 512,Đ
Đ
        [Description("A$5D AD$0C0 0 !D$>C²"•Ý
Table = 1024,Đ
Đ
        [Description("B$@C =D ?Cä@D$=Cä5 D @CT4D BC$>")]Đ
Vehicle = 2048,Đ
Đ
        [Description("A 2D$>C0>CÄ=D´9 D$@C =D ?Cä@D$5D "•Ý
Conveyor = 4096,Đ
Đ
        [Description("A0@CäGCT5")]Đ
Other = 8192Đ
}
</file>

```

```

<file path="Promatis.Net.MES.Domain.Interface/Interfaces/ITechnologicalOperation.cs">
namespace Promatis.Net.MES.Domain.Interface;Đ
Đ
public interface ITechnologicalOperationĐ
{Đ
    Đ
}
</file>

```

```

<file path="Promatis.Net.MES.Domain.Interface/Interfaces/ITechnologicalParameter.cs">
namespace Promatis.Net.MES.Domain.Interface;Đ
Đ
public interface ITechnologicalParameterĐ
{Đ
    Đ
}
</file>

```

```

<file path="Promatis.Net.MES.Domain.Interface/Promatis.Net.MES.Domain.Interface.csproj">
<Project Sdk="Microsoft.NET.Sdk">Đ
Đ
    <PropertyGroup>Đ
        <TargetFramework>net10.0</TargetFramework>Đ
        <ImplicitUsings>enable</ImplicitUsings>Đ
        <Nullable>enable</Nullable>Đ
    </PropertyGroup>Đ
Đ
    <ItemGroup>Đ
        <ProjectReference Include="..\..\Core\Domain.Interface\Promatis.Net.Domain.Interface.csproj" />Đ
    </ItemGroup>Đ
Đ
</Project>
</file>

```

```

<file path="Promatis.Net.MES.Domain/Promatis.Net.MES.Domain.csproj">
<Project Sdk="Microsoft.NET.Sdk">Đ
Đ
    <PropertyGroup>Đ
        <TargetFramework>net10.0</TargetFramework>Đ
        <ImplicitUsings>enable</ImplicitUsings>Đ
        <Nullable>enable</Nullable>Đ
    </PropertyGroup>Đ
Đ
    <ItemGroup>Đ

```

```

        <ProjectReference Include="...\Core\Domain\Promatis.Net.Domain.csproj" />
        <ProjectReference Include=".\
Promatis.Net.MES.Domain.Interface\Promatis.Net.MES.Domain.Interface.csproj" />
    </ItemGroup>
}
</Project>
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalOperation/TechnologicalOperationBase.cs">
using Promatis.Net.Domain;
using Promatis.Net.MES.Domain.Interface;
{
namespace Promatis.Net.MES.Domain;
{
/// <summary>
/// A 0Ct>C$0D0 BCTEC0>C'>C48Dt5D :C 0 Cä?CT@C FC,,0.
/// A00D ;CT4D45D" GC,,AD$>CR =CR04Cd5C05D 8C¢ 4CT@CT2Câ !B4 ABÂ CC 8D 0D0 :Cä=DD;C,,:D$K CÄ0C0?C,,=C40,
/// C, ?D 5CD>D BC 2C'0CTB D BD >C4> D$8C08Ct8D >C$0C0=D'9 C,,=D$5D DCT9D 4C'0 C 8Ct=CTA-C'>C48C¢8.D
/// </summary>
public abstract class TechnologicalOperationBase<T, TOLink, TPLink> : ReferenceTreeBase<T>, // <- A A,, "A,, 'AT!A
ITechnologicalOperation
where T : TechnologicalOperationBase<T, TOLink, TPLink>
where TOLink : class
where TPLink : class
{
    public bool IsLeaf { get; set; } = true;
}
    public virtual ICollection<TOLink> UnitLinks { get; set; } = new List<TOLink>();
    public virtual ICollection<TPLink> ParameterLinks { get; set; } = new List<TPLink>();
}
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalOperation/TechnologicalOperationBaseValidator.cs">
using FluentValidation;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Promatis.Net.MES.Domain.Interface;
{
namespace Promatis.Net.MES.Domain;
{
// A$0C'8CD0D$>D >C05D 0Dd8C, ,,>D BC 5D$AD0 1CT7 C,,7CÄ5C05C08C'•
public abstract class TechnologicalOperationBaseValidator<T> : ReferenceTreeBaseValidator<T>
where T : ReferenceTreeBase<T>, ITreeNode<T>, ITechnologicalOperation // <- A A,, "A,, 'AT!A AR B B A$ A
{
    protected TechnologicalOperationBaseValidator() : base()
    {
        // A 0Ct>C$KC' &VfW&Væ6T& 6Uf Æ-F F=" CCd5 C0@Cä2CT@C,,; Cä1D"8CR ?Cä;D0 ,, -BÂ æ ÖR•
        // At4CTADÂ <D² ?C,,HCT< C0@C 2C,,;C AD$@Cä3Cä 4C'0 D$5DT=Cä;Cä3C,,GCTAC¢8DR >C05D 0Dd8C•
    }
    RuleFor(x => x.Code)
        .NotEmpty()
        .WithMessage("A¢>CB >C05D 0Dd8C, >C 0Ct0D$5C'5C0â"•
        .MaxLength(50)
        .WithMessage("A¢>CB >C05D 0Dd8C, =CR 4Cä;Cd5C0 ?D 5C$KD,,0D$L 50 D 8CÄ2Cä;Cä2.");
}
}
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalOperation/
TechnologicalOperationHierarchyEngine.cs">
using System.Reflection;
using Promatis.Net.Domain.Interface;
{
namespace Promatis.Net.MES.Domain;
{
public static class TechnologicalOperationHierarchyEngine
{
/// <summary>
/// A4;Cä1C ;DÄ=D'9 C,,AD$>Dt=C,,: C0@C 2CDK: C0@Cä2CT@D05D"Â 2C ;C,,4C0> C'8 C$;Cä6CT=C,,5 CD>Dt5D =CT9 Cä?C
/// A00CÄ5D BC$> C ;Cä:C,,@D45D" ACä7CD0C08CR 2CTBCä: C$=D4BD 8 D$5D <C,,=C ;DÄ=D'E Cä?CT@C FC,,9 (IsLeaf =
/// B$8C08Ct0Dd8D0 ?CT@CT2CT4CT=C =C GC,,AD$KC' :Cä=D$@C :D" 4CT@CT2C BÂ GD$> C0>C'=CäAD$LDâ CD BD 0C00
/// </summary>
/// <typeparam name="T">A¢>C0:D 5D$=D'9 D$8C0 BCTEC0>C'>C48Dt5D :Cä9 Cä?CT@C FC,,8</typeparam>
/// <param name="parent">B >CD8D$5C'LD :C 0 Cä?CT@C FC,,0</param>
/// <param name="childIsLeaf">A0@C,,7C00C¢ BCä3CäÂ 0C$;D05D$AD0 ;C, ACä7CD0C$0CT<D'9/C05D 5CÄ5D"0CT<D'9 D
public static bool IsHierarchyValid<T>(T? parent, bool childIsLeaf)

```



```

    where T : class, ITreeNode<T>ð
{ð
    // 1. ATAC´8 D >CD8D$5C´O CÔ5D"Â <D² ACä7CD0CT< Cð>D =CT2Cä9 DÔ;CT<CT=D" r MD$> C$ACT3CD0 C$0C´8CD=Cä
    if (parent == null)ð
    {ð
        return true;ð
    }ð
ð
    // 2. Bt5D 5Cr @CTDC´5CðAC,,N (C,,;C, :C AD" : C,,=D$5D DCT9D C, CTAC´8 C K Cä= C KC²´ 8Ct2C´5Cð0CT< C0@
    // B$0C¢ :C : T CÔ0D ;CT4D45D$ADò >D" FV6†æöÆöv-6 Ä÷ W& F-öä& 6RÂ AC$>C"AD$2Cä -4ÆV b BC < C40D 0CÔBC
    PropertyInfo? isLeafProperty = typeof(T).GetProperty("IsLeaf");ð
    if (isLeafProperty != null)ð
    {ð
        bool parentIsLeaf = (bool)isLeafProperty.GetValue(parent)!;ð
        if (parentIsLeaf)ð
        {ð
            return false; // B$5D <C,,=C ;DÄ=D´9 D47CT; CÔ5 CÄ>Cd5D" 1D´BDÂ @Cä4C,,BCT;CT<ð
        }ð
    }ð
ð
    return true;ð
}ð
ð
    /// <summary>ð
    /// A0@Cä2CT@Dô5D"Â @C 7D 5D,,5CÔ> C´8 Cô5D 5Cð;DäGC BDÂ DC´0C2 -4ÆV b C D CD"5D BC$CDäICT9 Cä?CT@C FC,,8.D
    /// ATAC´8 D2 >Cô5D 0Dd8C, CCd5 DD8Ct8Dt5D :C, 5D BDÂ 4CäGCT@CÔ8CR 2CTBCð8 C" At#, C0@CT2D 0D"0D$L CTQ C
    /// </summary>ð
    public static bool CanChangeLeafStatus(bool hasChildren, bool requestedIsLeaf)ð
    {ð
        if (hasChildren && requestedIsLeaf)ð
        {ð
            return false;ð
        }ð
    }ð
ð
    return true;ð
}ð
}
</file>

```

```

<file path="Promatis.Net.MES.Domain/TechnologicalOperationParameter/
TechnologicalOperationParameterBase.cs">
using Promatis.Net.Domain;ð
using Promatis.Net.Domain.Interface;ð
using Promatis.Net.MES.Domain.Interface;ð
ð
namespace Promatis.Net.MES.Domain;ð
ð
/// <summary>ð
/// B 2Dô7D4ND"0Dò AD4ICÔ>D BDÂ ´ CÔ>C48CRÔ:Cäô<CÔ>C48CĚ² <CT6CDC D$5DT=Cä;Cä3C,,GCTACð8CÄ8 Cä?CT@C FC,,0CÄ8 C,
/// Aä?D 5CD5C´OCTB D$@CT1Cä2C =C,,0 C¢ 7C ?Cä;CÔ5CÔ8Dâ ?C @C <CTBD >C" 4C´O Cð>CÔ:D 5D$=Cä9 Cä?CT@C FC,,8.D
/// </summary>ð
public abstract class TechnologicalOperationParameterBase<TOperation, TParameter> : DomainObject,
ISoftDeletableð
    where TOperation : DomainObject, ITechnologicalOperationð
    where TParameter : DomainObject, ITechnologicalParameterð
{ð
    public Guid OperationId { get; set; }ð
ð
    /// <summary>ð
    /// A00C$8C40Dd8Cä=CÔ>CR AC$>C"AD$2Cä : C 0Ct>C$>C´ >Cô5D 0Dd8C,i
    /// </summary>ð
    public virtual TOperation Operation { get; set; } = null!;ð
ð
    public Guid ParameterId { get; set; }ð
ð
    /// <summary>ð
    /// A00C$8C40Dd8Cä=CÔ>CR AC$>C"AD$2Cä : C 0Ct>C$>CÄC C00D 0CÄ5D$@D2i
    /// </summary>ð
    public virtual TParameter Parameter { get; set; } = null!;ð
ð
    /// <summary>ð
    /// A0@C,,7CÔ0C¢ BCä3CäÂ GD$> C00D 0CÄ5D$@ Cä1Dô7C BCT;CT= CD;Dò 7C ?Cä;CÔ5CÔ8Dò1
    /// </summary>ð
    public bool IsRequired { get; set; } = false;ð
ð
    /// <summary>ð

```

```

    /// AÔ>CÄ8CÔ0C´LCÔ>CR 7CÔ0Dt5CÔ8CR ?C @C <CTBD 0 CÔ> D4<Cä;Dt0CÔ8Däí
    /// </summary>ð
    public string? DefaultValue { get; set; }ð
ð
    /// <summary>ð
    /// AÄ8CÔ8CÄ0C´LCÔ> CD>CÔCD BC,,<Cä5 Ct=C GCT=C,,5.ð
    /// </summary>ð
    public double? MinValue { get; set; }ð
ð
    /// <summary>ð
    /// AÄ0CðAC,,<C ;DÄ=Cä 4Cä?D4AD$8CÄ>CR 7CÔ0Dt5CÔ8CRí
    /// </summary>ð
    public double? MaxValue { get; set; }ð
ð
    public DateTime? DeletedAt { get; set; }ð
    public string? DeletedBy { get; set; }ð
}
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalOperationParameter/
TechnologicalOperationParameterBaseValidator.cs">
using FluentValidation;ð
using Promatis.Net.Domain;ð
using Promatis.Net.MES.Domain.Interface;ð
ð
namespace Promatis.Net.MES.Domain;ð
ð
public abstract class TechnologicalOperationParameterBaseValidator<T, TOperation, TParameter> :
DomainObjectValidator<T>ð
    where T : TechnologicalOperationParameterBase<TOperation, TParameter>ð
    where TOperation : DomainObject, ITechnologicalOperationð
    where TParameter : TechnologicalParameterBaseð
{ð
    protected TechnologicalOperationParameterBaseValidator()ð
    {ð
        RuleFor(x => x.OperationId)ð
            .NotEmpty()ð
            .WithMessage("A,,4CT=D$8DD8Cð0D$>D >Cô5D 0Dd8C, >C 0ct0D$5C´5C0â""½
ð
        RuleFor(x => x.ParameterId)ð
            .NotEmpty()ð
            .WithMessage("A,,4CT=D$8DD8Cð0D$>D BCTECÔ>C´>C48Dt5D :Cä3Cä ?C @C <CTBD 0 Cä1Dô7C BCT;CT=.");ð
ð
        // Að@CäAD Ô?Cä;CT2C 0 C$0C´8CD0Dd8Dôç C :D 8CÄ0C´LCÔ>CR 7CÔ0Dt5CÔ8CR =CR <Cä6CTB C KD$L CÄ5CÔLD,,5 C
        RuleFor(x => x.MaxValue)ð
            .GreaterThanOrEqualTo(x => x.MinValue)ð
            .When(x => x.MinValue.HasValue && x.MaxValue.HasValue)ð
            .WithMessage("AÄ0CðAC,,<C ;DÄ=Cä 4Cä?D4AD$8CÄ>CR 7CÔ0Dt5CÔ8CR =CR <Cä6CTB C KD$L CÄ5CÔLD,,5 CÄ8CÔ8C
    }ð
}
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalOperationUnit/TechnologicalOperationUnitBase.cs">
using Promatis.Net.Domain;ð
using Promatis.Net.Domain.Interface;ð
using Promatis.Net.MES.Domain.Interface;ð
ð
namespace Promatis.Net.MES.Domain;ð
ð
/// <summary>ð
/// A 1D BD 0CðBCÔ0Dò 1C 7C 4C´O D 2Dô7C, ,,@C AD,,8D OCT<C 0>ð
/// </summary>ð
public abstract class TechnologicalOperationUnitBase<TOperation> : DomainObject, ISoftDeletableð
    where TOperation : DomainObject, ITechnologicalOperationð
{ð
    public Guid OperationId { get; set; }ð
ð
    /// <summary>ð
    /// A00C$8C40Dd8Cä=CÔ>CR AC$>C"AD$2Cä : Cä?CT@C FC,,8 (C CCD5D" 7C :D KD$> Cð>CÔ:D 5D$=D´< D$8CÔ>CÄ 2 MDM)
    /// </summary>ð
    public virtual TOperation Operation { get; set; } = null!;ð
ð
    public Guid UnitId { get; set; }ð
ð
    /// <summary>ð
    /// Aô@Dô<C 0 D AD´;Cð0 CÔ0 C 0ct>C$KC´ :C´0D A Cä1Cä@D44Cä2C =C,,0. ð

```

```

    /// EF Core C 2D$>CÄ0D$8Dt5D :C, AC$0Cd5D" MD$> D BC 1C´8Dd5C´ >C >D CCD>C$0C08Dò ?Câ ACä3C´0D,,5C08Dô<.>
    /// </summary>ð
    public virtual UnitBase Unit { get; set; } = null!;ð
ð
    public int Priority { get; set; } = 1;ð
ð
    public DateTime? DeletedAt { get; set; }ð
    public string? DeletedBy { get; set; }ð
}
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalOperationUnit/
TechnologicalOperationUnitBaseValidator.cs">
using FluentValidation;ð
using Promatis.Net.Domain;ð
using Promatis.Net.MES.Domain.Interface;ð
ð
namespace Promatis.Net.MES.Domain;ð
ð
/// <summary>ð
/// A 0Ct>C$KC´ 2C ;C,,4C BCä@ D 2Dô7C•
/// </summary>ð
public abstract class TechnologicalOperationUnitBaseValidator<T, TOperation> :
DomainObjectValidator<T>ð
    where T : TechnologicalOperationUnitBase<TOperation>ð
    where TOperation : DomainObject, ITechnologicalOperationð
{ð
    protected TechnologicalOperationUnitBaseValidator()ð
    {ð
        RuleFor(x => x.OperationId)ð
            .NotEmpty()ð
            .WithMessage("A,,4CT=D$8DD8Cð0D$>D >Cô5D 0Dd8C, >C 0Ct0D$5C´5C0â""½
ð
        RuleFor(x => x.UnitId)ð
            .NotEmpty()ð
            .WithMessage("A,,4CT=D$8DD8Cð0D$>D >C >D CCD>C$0C08Dò >C 0Ct0D$5C´5C0â""½
ð
        RuleFor(x => x.Priority)ð
            .InclusiveBetween(1, 10)ð
            .WithMessage("Aô@C,,>D 8D$5D" >C >D CCD>C$0C08Dò 4Cä;Cd5Cò 1D´BDÂ 2 CD8C ?C 7Cä=CR >D" 4Cä ä""
    }ð
}
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalParameter/TechnologicalParameterBase.cs">
using Promatis.Net.Domain;ð
using Promatis.Net.MES.Domain.Interface;ð
ð
namespace Promatis.Net.MES.Domain;ð
ð
/// <summary>ð
/// A 0Ct>C$KC´ BCTECô>C´>C48Dt5D :C,,9 Cò0D 0CÄ5D$@ (D ?D 0C$>Dt=C,,: DT0D 0CðBCT@C,,AD$8Cð ?D >Dd5D ACä2).ð
/// </summary>ð
public abstract class TechnologicalParameterBase : ReferenceBase, ITechnologicalParameterð
{ð
    /// <summary>ð
    /// A$=CTHC08C´ :C´Ndr =C ACô@C 2CäGC08Cð 5CD8C08Db 8Ct<CT@CT=C,,0.ð
    /// </summary>ð
    public Guid? UnitOfMeasurementId { get; set; }ð
ð
    public virtual UnitOfMeasurement? UnitOfMeasurement { get; set; }ð
ð
    /// <summary>ð
    /// B$8Cò 4C =C0KDR ?C @C <CTBD 0 (C00C0@C,,<CT@: Bt8D ;CâÂ !D$@Cä:C Â D4;CT2Cä´í
    /// </summary>ð
    public string DataType { get; set; } = "Numeric";ð
ð
    /// <summary>ð
    /// B 0Ct@CTHCT=C0KCR <CTBCä4D² @C AD GE BC
    /// </summary>ð
    public CalculationMethod AllowedMethods { get; set; } = CalculationMethod.None;ð
}
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalParameter/TechnologicalParameterBaseValidator.cs">
using FluentValidation;ð

```

```

using Promatis.Net.Domain;
namespace Promatis.Net.MES.Domain;
public abstract class TechnologicalParameterBaseValidator<T> : ReferenceBaseValidator<T>
    where T : TechnologicalParameterBase
{
    private static readonly string[] AllowedDataTypes = ["Numeric", "String", "Boolean",
"DateTime"];
    protected TechnologicalParameterBaseValidator() : base()
    {
        // 1. A$0C'8CD0Dd8D0 5CD8C08DdK C,,7CÄ5D 5C08D0 „4CäAD$CC0=C Â BC : C0C¢ B r MD$> TechnologicalParame
RuleFor(x => x.UnitOfMeasurement)
    .NotNull()
    .WithMessage("AT4C,,=C,,FC 8Ct<CT@CT=C,,0 C05 CÄ>Cd5D" 1D'BDÄ çVÆÄâ""½
    // 2. A$0C'8CD0Dd8D0 BC,,?C 4C =C0KD]
RuleFor(x => x.DataType)
    .NotEmpty()
    .WithMessage("B$8C0 4C =C0KDR ?C @C <CTBD 0 Cä1D07C BCT;CT=.")
    .Must(type => AllowedDataTypes.Contains(type))
    .WithMessage($"A05CD>C0CD BC,,<D'9 D$8C0 4C =C0KDRâ C 7D 5D,,5C0=D'5 Ct=C GCT=C,,0: {string.Join(",
    }
}
</file>

```

```

<file path="Promatis.Net.MES.Domain/TechnologicalParameterCalcMethod/
TechnologicalParameterCalcMethodBase.cs">
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Promatis.Net.MES.Domain.Interface;
namespace Promatis.Net.MES.Domain;
public abstract class TechnologicalParameterCalcMethodBase<TUnit, TOperation, TParameter> :
DomainObject, ISoftDeletable
    where TUnit : UnitBase
    where TOperation : DomainObject, ITechnologicalOperation
    where TParameter : TechnologicalParameterBase
{
    public Guid UnitId { get; set; }
    /// <summary>
    /// Bd5DT>C$0D0 5CD8C08Dd0
    /// </summary>
    public virtual TUnit Unit { get; set; } = null!;
    public Guid TechnologicalOperationId { get; set; }
    /// <summary>
    /// B$5DT=Cä;Cä3C,,GCTAC0D0 >C05D 0Dd8Dý
    /// </summary>
    public virtual TOperation TechnologicalOperation { get; set; } = null!;
    public Guid TechnologicalParameterId { get; set; }
    /// <summary>
    /// B$5DT=Cä;Cä3C,,GCTAC08C' ?C @C <CTBD
    /// </summary>
    public virtual TParameter TechnologicalParameter { get; set; } = null!;
    /// <summary>
    /// AÄ5D$>CB @C ADt5D$0
    /// </summary>
    public CalculationMethod CalculationMethod { get; set; } = CalculationMethod.None;
    public DateTime? DeletedAt { get; set; }
    public string? DeletedBy { get; set; }
}
</file>

```

```

<file path="Promatis.Net.MES.Domain/TechnologicalParameterCalcMethod/
TechnologicalParameterCalcMethodBaseValidator.cs">
using FluentValidation;
using Promatis.Net.Domain;

```

```

using Promatis.Net.MES.Domain.Interface;
{
namespace Promatis.Net.MES.Domain;
{
public abstract class TechnologicalParameterCalcMethodBaseValidator<T, TUnit, TOperation,
TParameter> : DomainObjectValidator<T>
    where T : TechnologicalParameterCalcMethodBase<TUnit, TOperation, TParameter>
    where TUnit : UnitBase
    where TOperation : DomainObject, ITechnologicalOperation
    where TParameter : TechnologicalParameterBase
{
    protected TechnologicalParameterCalcMethodBaseValidator()
    {
        RuleFor(x => x.UnitId)
            .NotEmpty().WithMessage("A,,4CT=D$8DD8C=0D$>D FCTECä2Cä9 CT4C,,=C,,FD² >C 0Ct0D$5C´5C0â""½
D
        RuleFor(x => x.TechnologicalOperationId)
            .NotEmpty().WithMessage("A,,4CT=D$8DD8C=0D$>D BCTECÔ>C´>C48Dt5D :Cä9 Cä?CT@C FC,,8 Cä1Dô7C BCT;CT=
D
        RuleFor(x => x.TechnologicalParameterId)
            .NotEmpty().WithMessage("A,,4CT=D$8DD8C=0D$>D BCTECÔ>C´>C48Dt5D :Cä3Câ ?C @C <CTBD 0 Cä1Dô7C BCT;
D
        // Aô@Cä2CT@Dô5CÄÂ GD$> CÄ5D$>CB @C ADt5D$0 C KC² OC$=Cä 2D´1D 0C0 ?Cä;DÄ7Cä2C BCT;CT<D
        RuleFor(x => x.CalculationMethod)
            .NotEqual(CalculationMethod.None)
            .WithMessage("A05Cä1DT>CD8CÄ> C$KC @C BDÄ :Cä@D 5C=BC0KC´ <CTBCä4 D 0D GCTBC ?C @C <CTBD >C"â""½
    }
}
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalParameterValue/TechnologicalParameterValueBase.cs">
using Promatis.Net.Domain;
{
namespace Promatis.Net.MES.Domain;
{
public abstract class TechnologicalParameterValueBase<TUnit, TParameter> : DomainObject
    where TUnit : UnitBase
    where TParameter : TechnologicalParameterBase
{
    public Guid UnitId { get; set; }
D
    /// <summary>D
    /// Bd5DT>C$0D0 5CD8C08Dd0 (C,,AD$>Dt=C,,: CD0C0=D´E)D
    /// </summary>D
    public virtual TUnit Unit { get; set; } = null!;D
D
    public Guid TechnologicalParameterId { get; set; }D
D
    /// <summary>D
    /// B$5DT=Cä;Cä3C,,GCTAC=8C´ ?C @C <CTBD
    /// </summary>D
    public virtual TParameter TechnologicalParameter { get; set; } = null!;D
D
    /// <summary>D
    /// B KD >CR ,3D 0Ct=Cä5") Ct=C GCT=C,,5, C05D 5CD0C0=Cä5 CäB SCADA/IoT C,,;C, >C05D 0D$>D 0D
    /// </summary>D
    public string Value { get; set; } = string.Empty;D
D
    /// <summary>D
    /// B$>Dt=Cä5 C$@CT<D0 DC,,:D 0Dd8C, ?C @C <CTBD 0 (Time-Series CÄ5D$:C •
    /// </summary>D
    public DateTime Date { get; set; } = DateTime.UtcNow;D
}
}
</file>

<file path="Promatis.Net.MES.Domain/TechnologicalParameterValue/
TechnologicalParameterValueBaseValidator.cs">
using FluentValidation;
using Promatis.Net.Domain;
{
namespace Promatis.Net.MES.Domain;
{
public abstract class TechnologicalParameterValueBaseValidator<T, TUnit, TParameter> :
DomainObjectValidator<T>
    where T : TechnologicalParameterValueBase<TUnit, TParameter>D
    where TUnit : UnitBaseD

```

```

where TParameter : TechnologicalParameterBase
{
    protected TechnologicalParameterValueBaseValidator()
    {
        RuleFor(x => x.UnitId)
            .NotEmpty().WithMessage("A,,4CT=D$8DD8C=D$>D FCTECä2Cä9 CT4C,,=C,,FD² >C 0Ct0D$5C´5C0â""½

        RuleFor(x => x.TechnologicalParameterId)
            .NotEmpty().WithMessage("A,,4CT=D$8DD8C=D$>D BCTECÔ>C´>C48Dt5D :Cä3Câ ?C @C <CTBD 0 Cä1D07C BCT;

        RuleFor(x => x.Value)
            .NotNull().WithMessage("At=C GCT=C,,5 C00D 0CÄ5D$@C =CR <Cä6CTB C KD$L null.")
            // B 0Ct@CTHC 5CÄ ?D4AD$CDâ AD$@Cä:D2Â BC : C=0C¢ 4C´0 String-C00D 0CÄ5D$@Cä2 D0BCâ <Cä6CTB C KD$
            // C0> D41C,,@C 5CÄ AC´CDt0C"=D´5 C0@Cä1CT;D² ?Câ :D 0D0<D
            .Must(val => val == null || val.Trim() == val)
            .WithMessage("At=C GCT=C,,5 C05 CÄ>Cd5D" =C GC,,=C BDÄAD0 8C´8 Ct0C=0C0GC,,2C BDÄAD0 ?D >C 5C´0CÄ8."

        RuleFor(x => x.Date)
            .Must(date => date <= DateTime.UtcNow.AddSeconds(5))
            .WithMessage("A$@CT<D0 DC,,:D 0Dd8C, ?C @C <CTBD 0 C05 CÄ>Cd5D" 1D´BDÂ 2 C CCDCD"5CÄâ""½
    }
}
</file>

```

```

<file path="Promatis.Net.MES.Domain/UnitBase/UnitBase.cs">
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Promatis.Net.MES.Domain.Interface;
namespace Promatis.Net.MES.Domain;
/// <summary>
/// A 1D BD 0C=BC00D0 1C 7C 4C´0 C$ACTE Cä@C40C08Ct0Dd8Cä=C0KDR 8 C0@Cä8Ct2Cä4D BC$5C0=D´E CT4C,,=C,,F.D
/// </summary>
public abstract class UnitBase : ReferenceTreeBase<UnitBase>, IAudit
{
    public UnitKind Kind { get; init; }
    public required UnitType Type { get; set; }
}
</file>

```

```

<file path="Promatis.Net.MES.Domain/UnitBase/UnitBaseValidator.cs">
using FluentValidation;
using Promatis.Net.Domain;
namespace Promatis.Net.MES.Domain;
/// <summary>
/// A 1D BD 0C=BC0KC´ 2C ;C,,4C BCä@ CD;D0 1C 7Cä2Cä3Câ :C´0D AC Væ-D& 6R1
/// </summary>
/// <typeparam name="T">B$8C0Ä =C AC´5CDCCT<D´9 CäB UnitBase.</typeparam>
public abstract class UnitBaseValidator<T> : ReferenceBaseValidator<T>
    where T : UnitBase
{
    protected UnitBaseValidator() : base()
    {
        // A0@Cä2CT@C=0, DtBCâ 2 Type C05 C0@C,,HC´> C0CD BCä5/CD5DD>C´BC0>CR 7C00Dt5C08CR DC´0C4>C-
        RuleFor(x => x.Type).NotEmpty();

        // A0@Cä2CT@C=0 D >CäBC$5D$AD$2C,,0 A=0D$5C4>D 8C, „¶-æB´ 8 B$8C00 (Type) C00 CäAC0>C$5 C 8D$>C$KDR <C
        RuleFor(x => x)
            .Must(u => ((int)u.Kind & (int)u.Type) != 0)
            .WithMessage("B$8C0 =CR ACä>D$2CTBD BC$CCTB C=0D$5C4>D 8C,â""½
    }
}
</file>

```

```

<file path="Promatis.Net.MES.Domain/UnitBase/UnitHierarchyEngine.cs">
using Promatis.Net.MES.Domain.Interface;
namespace Promatis.Net.MES.Domain;
public static class UnitHierarchyEngine
{
    /// <summary>
    /// A4;Cä1C ;DÄ=D´9 C,,AD$>Dt=C,,: C0@C 2CDK: C0@Cä2CT@D05D"Â 2C ;C,,4C0> C´8 C$;Cä6CT=C,,5 D 5C 5C0:C 2 D >

```

```

/// A,,ACô>C`LctCCTBD O CÔ0CÄ5D BC$> C@0C¢ 2 B #A -D$@C,,3C45D 0DRÂ BC : C, 2 UI-C$0C´8CD0Dd8C,í
/// </summary>ð
public static bool IsHierarchyValid(UnitKind parentKind, UnitKind childKind)ð
{ð
    // 1. B$5D <C,,=C ;DÄ=D´9 D47CT; Position CÔ8C¤>C44C =CR <Cä6CTB C KD$L D >CD8D$5C´5Cí
    if (parentKind == UnitKind.Position) return false;ð
ð
    // 2. AD5C00D BC <CT=D" <Cä6CTB D >CD5D 6C BDÂ 2D Q, C@Cä<CR ÷6-F-öí
    if (parentKind == UnitKind.Department) return childKind != UnitKind.Position;ð
ð
    // 3. B ?CTFC,,0C´8ct8D >C$0CÔ=D´5 Ct>CÔK C,,7Cä;C,,@Cä2C =D² 4D CC2 >D" 4D CC40, CÔ> CÄ>C4CD" ACä4CT@Cd
    if (parentKind is UnitKind.Production or UnitKind.Transport or UnitKind.Storage)ð
    {ð
        return childKind == parentKind || childKind == UnitKind.Position;ð
    }ð
ð
    return true;ð
}ð
ð
/// <summary>ð
/// A$>Ct2D 0D"0CTB D ?C,,ACä: Aä Aä B AÔ B´% D$8Cô>C" ...Væ-EG- R´ 4C´O C$KC00CD0DäICT3Cä ACô8D :C 2 UI,ð
/// CäACÔ>C$KC$0DôADÄ =C C BCT3Cä@C,,8 (UnitKind) D >CD8D$5C´LD :Cä3Cä CCT;C Ä 2 C¤>D$>D KC´ <D² ACT9Dt0
/// </summary>ð
public static IEnumerable<UnitType> GetAllowedChildTypes(UnitKind parentKind)ð
{ð
    var allTypes = (UnitType[])Enum.GetValues(typeof(UnitType));ð
    var allowedTypes = new List<UnitType>();ð
ð
    foreach (UnitType type in allTypes)ð
    {ð
        if (type == UnitType.None) continue;ð
ð
        // 1. B =C GC ;C =C ECä4C,,<, C¢ :C :Cä<D2 ?CäBCT=Dd8C ;DÄ=Cä<D2 ¶-æB >D$=CäAC,,BD O DÔBCäB D$8Côi
        // B :C =C,,@D45CÄ 2D 5 C¤0D$5C4>D 8C,Ä GD$>C K Cô>CÔ0D$L, C" :C :D4N CÄ0D :D2 ?Cä?C 4C 5D" MD$>D"
        foreach (UnitKind potentialChildKind in (UnitKind[])Enum.GetValues(typeof(UnitKind)))ð
        {ð
            // ATAC´8 D$8Cò ?Cä1C,,BCä2Cä 2DT>CD8D" 2 DÔBD2 :C BCT3Cä@C,,Nð
            if (((int)potentialChildKind & (int)type) != 0)ð
            {ð
                // A, 5D ;C, ?D 0C$8C´0 C,,5D 0D EC,,8 B At AT(A .B" 2C´>Cd8D$L DÔBD2 :C BCT3Cä@C,,N C" BCT
                if (IsHierarchyValid(parentKind, potentialChildKind))ð
                {ð
                    if (!allowedTypes.Contains(type))ð
                    {ð
                        allowedTypes.Add(type);ð
                    }ð
                }ð
            }ð
        }ð
    }ð
}ð
ð
return allowedTypes;ð
}
}
</file>

<file path="Promatis.Net.MES.Domain/UnitOfMeasurement/UnitOfMeasurement.cs">
using Promatis.Net.Domain;ð
ð
namespace Promatis.Net.MES.Domain;ð
ð
/// <summary>ð
/// AD>CÄ5C0=C O D CD"=CäAD$L CT4C,,=C,,FD² 8Ct<CT@CT=C,,O (AÔ!A,´ 4C´O D$5DT=Cä;Cä3C,,GCTAC¤8DR ?C @C <CTBD >C"
/// </summary>ð
public class UnitOfMeasurement : ReferenceBaseð
{ð
    // B 2Cä9D BC$0 Id, Code, Name, Description CÔ0D$8C$=Cä CCÔ0D ;CT4Cä2C =D² >D" &VfW&Væ6T& 6Rí
    // Code - C¤@C BC¤>CR >C >Ct=C GCT=C,,5 (CÔ0Cô@C,,<CT@, "°C", "CÄ<", "Cä1/CÄ8C0"´í
    // Name - Cô>C´=Cä5 CÔ0C,,<CT=Cä2C =C,,5 (CÔ0Cô@C,,<CT@, "A4@C 4D4A Bd5C´LD 8Dò"Ä $ C,,;C´8CÄ5D$@").ð
}
</file>

<file path="Promatis.Net.MES.Domain/UnitOfMeasurement/UnitOfMeasurementValidator.cs">
using FluentValidation;ð
using Promatis.Net.Domain;ð
ð

```

```

namespace Promatis.Net.MES.Domain;
{
    public class UnitOfMeasurementValidator : ReferenceBaseValidator<UnitOfMeasurement>
    {
        // A05D 5Cä?D 5CD5C'0CT< C 0Ct>C$CDâ @CT3D4;D0@C=C C00 D 0D HC,,@CT=C0CDâ ,,@D4A/C'0D"Â FC,,DD K, C0@Cä1CT;D
        protected override string CodeRegexPattern => @"^[\p{L}\p{N}_\.\°\/\*\-\s\%]*$";
        protected override string CodeValidationMessage => "A@C BC@>CR >C >Ct=C GCT=C,,5 CÄ>Cd5D" ACä4CT@Cd0D$L C
        public UnitOfMeasurementValidator() : base()
        {
            // A0@C 2C,,;Câ æ÷DV× G' 8 Matches D46CR >D$@C 1CäBC ND" 2 C 0Ct>C$>CÂ :C'0D ACR ?Câ =C HCT<D2 =Cä2Cä<
            RuleFor(x => x.Code)
                .NotEmpty().WithMessage("A@C BC@>CR >C >Ct=C GCT=C,,5 Cä1D07C BCT;DÄ=Câ 4C'0 Ct0C0>C'=CT=C,,0.")
                .MaxLength(15).WithMessage("A@C BC@>CR >C >Ct=C GCT=C,,5 C05 CD>C'6C0> C0@CT2D'HC BDÄ R AC,,<
            RuleFor(x => x.Name)
                .MaxLength(100).WithMessage("A0>C'=Cä5 C00C,,<CT=Cä2C =C,,5 CT4C,,=C,,FD² 8Ct<CT@CT=C,,0 C05 CD>C'
        }
    }
}
</file>

<file path="Promatis.Net.MES.Service/Promatis.Net.MES.Service.csproj">
<Project Sdk="Microsoft.NET.Sdk">
{
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
    </PropertyGroup>
{
    <ItemGroup>
        <ProjectReference Include="..\..\Core\Service\Promatis.Net.Service.csproj" />
        <ProjectReference Include="..\Promatis.Net.MES.Data\Promatis.Net.MES.Data.csproj" />
    </ItemGroup>
}
</Project>
</file>

<file path="Promatis.Net.MES.Service/TechnologicalOperationParameterService/
ITechnologicalOperationParameterService.cs">
using Promatis.Net.Domain;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;
using Promatis.Net.Service;
{
    namespace Promatis.Net.MES.Service;
    {
        public interface ITechnologicalOperationParameterService<T, TOperation, TParameter> :
        IBaseService<T, Guid>
        {
            where T : TechnologicalOperationParameterBase<TOperation, TParameter>
            where TOperation : DomainObject, ITechnologicalOperation
            where TParameter : TechnologicalParameterBase
        }
        {
            /// <summary>
            /// A0>C'CDt8D$L C$ACR ?C @C <CTBD K, C0@C,,2D07C =C0KCR : C@>C0:D 5D$=Cä9 Cä?CT@C FC,,8D
            /// </summary>
            Task<List<T>> GetByOperationIdAsync(Guid operationId);
        }
        {
            /// <summary>
            /// A0>C'CDt8D$L D$>C'LC@> Cä1D07C BCT;DÄ=D'5 C00D 0CÄ5D$@D² 4C'0 Cä?CT@C FC,,8D
            /// </summary>
            Task<List<T>> GetRequiredParametersByOperationIdAsync(Guid operationId);
        }
    }
}
</file>

<file path="Promatis.Net.MES.Service/TechnologicalOperationParameterService/
TechnologicalOperationParameterService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;
using Promatis.Net.Service;
{
    namespace Promatis.Net.MES.Service;

```



```

    0
public abstract class TechnologicalOperationParameterService<T, TOperation, TParameter, TContext>(0
    IDbContextFactory<TContext> contextFactory)0
    : BaseService<T, Guid, TContext>(contextFactory), ITechnologicalOperationParameterService<T,
TOperation, TParameter>0
    where T : TechnologicalOperationParameterBase<TOperation, TParameter>0
    where TOperation : DomainObject, ITechnologicalOperation0
    where TParameter : TechnologicalParameterBase0
    where TContext : DbContext0
{0
    public async Task<List<T>> GetByOperationIdAsync(Guid operationId)0
    {0
        await using TContext context = await ContextFactory.CreateDbContextAsync();0
    0
        return await context.Set<T>().0
            .Where(x => x.OperationId == operationId)0
            .Include(x => x.Parameter) // B @C 7D2 ?Cä4D$0C48C$0CT< D ?D 0C$>Dt=C,,: Cö0D 0CÄ5D$@Cä2 CD;Dò T•
            .AsNoTracking()0
            .ToListAsync();0
    }0
    0
    public async Task<List<T>> GetRequiredParametersByOperationIdAsync(Guid operationId)0
    {0
        await using TContext context = await ContextFactory.CreateDbContextAsync();0
    0
        return await context.Set<T>().0
            .Where(x => x.OperationId == operationId && x.IsRequired)0
            .Include(x => x.Parameter)0
            .AsNoTracking()0
            .ToListAsync();0
    }0
}
</file>

```

```

<file path="Promatis.Net.MES.Service/TechnologicalOperationService/
ITechnologicalOperationService.cs">
using Promatis.Net.Domain;0
using Promatis.Net.MES.Domain;0
using Promatis.Net.MES.Domain.Interface;0
using Promatis.Net.Service;0
0
namespace Promatis.Net.MES.Service;0
0
public interface ITechnologicalOperationService<T, TLink> : ITreeService<T>0
    where T : ReferenceTreeBase<T>, ITechnologicalOperation,
Promatis.Net.Domain.Interface.IDomainObjectHasKey<Guid>0
    where TLink : class0
{0
    /// <summary>0
    /// Að>C´CDt8D$L D ?C,,ACä: D 0Ct@CTHCT=CÔ>C4> Cä1Cä@D44Cä2C =C,,0 (Dä=C,,BCä2) CD;Dò :Cä=C@CTBCÔ>C´ BCTECÔ
    /// </summary>0
    Task<List<UnitBase>> GetAllowedUnitsAsync(Guid operationId);0
}
</file>

```

```

<file path="Promatis.Net.MES.Service/TechnologicalOperationService/
TechnologicalOperationService.cs">
using Microsoft.EntityFrameworkCore;0
using Promatis.Net.Domain;0
using Promatis.Net.Domain.Interface;0
using Promatis.Net.MES.Domain;0
using Promatis.Net.MES.Domain.Interface;0
using Promatis.Net.Service;0
0
namespace Promatis.Net.MES.Service;0
0
public abstract class TechnologicalOperationService<T, TLink, TContext>(IDbContextFactory<TContext>
contextFactory)0
    : ReferenceTreeService<T, TContext>(contextFactory), ITechnologicalOperationService<T, TLink>0
    where T : ReferenceTreeBase<T>, ITechnologicalOperation, IDomainObjectHasKey<Guid>, new()0
    where TLink : TechnologicalOperationUnitBase<T>0
    where TContext : DbContext0
{0
    public async Task<List<UnitBase>> GetAllowedUnitsAsync(Guid operationId)0
    {0
        await using TContext context = await ContextFactory.CreateDbContextAsync();0
    0
    }
}

```

```

Ð
    // LINQ-Ct0C0@CäA CäBD 0C 0D$KC$0CTB C,,4CT0C`LC0>:Ð
    // A ;C 3Cä4C @D0 BCä<D2Â GD$> C" FV6†æ0Æöv-6 Ä÷ W& F-öäVæ-D& 6R AC$>C"AD$2Cä Væ-B
    // C,,<CT5D" AD$@Cä3C,,9 D$8C0 Væ-D& 6RÂ Tb 6÷&R ?CäAD$@Cä8D" GC,,AD$KC' 5 Â0=0"â 0C$BCä<C BC,,GCTAC=8.Ð
    return await context.Set<TLink>()Ð
        .Where(x => x.OperationId == operationId)Ð
        .Select(x => x.Unit)Ð
        .AsNoTracking()Ð
        .ToListAsync();Ð
}Ð
Ð
    /// <summary>Ð
    /// AäAD$0C$;D05CÄ <CTBCä4 C 1D BD 0C=BC0KCÄ =C BCT:D4ICT< D4@Cä2C05. A= >C0:D 5D$=D´9 C0@C,,:C´0CD=Cä9 D
    /// D$5DT?D >Dd5D ACä2 D 0CÄ ?CT@CT>C0@CT4CT;C,,B CT3Cä 4C´O D >Ct4C =C,,O D 2Cä8DR ?Cä;C,,<Cä@DD=D´E Cä?CT@
    /// </summary>Ð
    public abstract override Task<T> CreateChildTemplateAsync(T parent);Ð
}
</file>

<file path="Promatis.Net.MES.Service/TechnologicalParameterCalcMethodService/
ITechnologicalParameterCalcMethodService.cs">
using Promatis.Net.Domain;Ð
using Promatis.Net.MES.Domain;Ð
using Promatis.Net.MES.Domain.Interface;Ð
using Promatis.Net.Service;Ð
Ð
namespace Promatis.Net.MES.Service;Ð
Ð
public interface ITechnologicalParameterCalcMethodService<T, TUnit, TOperation, TParameter> :
IBaseService<T, Guid>Ð
    where T : TechnologicalParameterCalcMethodBase<TUnit, TOperation, TParameter>Ð
    where TUnit : UnitBaseÐ
    where TOperation : DomainObject, ITechnologicalOperationÐ
    where TParameter : TechnologicalParameterBaseÐ
{Ð
    /// <summary>Ð
    /// A00C"BC, :Cä=C=0CTBC0CDä 8C0AD$@D4:Dd8Dä @C ADt5D$0 CD;D0 BD >C":Cf¢ C >D CCD>C$0C08CR ² C05D 0Dd8D
    /// </summary>Ð
    Task<T?> GetCalcMethodAsync(Guid unitId, Guid operationId, Guid parameterId);Ð
Ð
    /// <summary>Ð
    /// A0>C´CDt8D$L C$ACR <CTBCä4D² @C ADt5D$0 CD;D0 :Cä=C=0CTBC0>C´ >C05D 0Dd8C, =C >C >D CCD>C$0C08C•
    /// </summary>Ð
    Task<List<T>> GetMethodsByOperationAndUnitAsync(Guid operationId, Guid unitId);Ð
}
</file>

<file path="Promatis.Net.MES.Service/TechnologicalParameterCalcMethodService/
TechnologicalParameterCalcMethodService.cs">
using Microsoft.EntityFrameworkCore;Ð
using Promatis.Net.Domain;Ð
using Promatis.Net.MES.Domain;Ð
using Promatis.Net.MES.Domain.Interface;Ð
using Promatis.Net.Service;Ð
Ð
namespace Promatis.Net.MES.Service;Ð
Ð
public abstract class TechnologicalParameterCalcMethodService<T, TUnit, TOperation, TParameter,
TContext>(Ð
    IDbContextFactory<TContext> contextFactory)Ð
    : BaseService<T, Guid, TContext>(contextFactory), ITechnologicalParameterCalcMethodService<T,
TUnit, TOperation, TParameter>Ð
    where T : TechnologicalParameterCalcMethodBase<TUnit, TOperation, TParameter>Ð
    where TUnit : UnitBaseÐ
    where TOperation : DomainObject, ITechnologicalOperationÐ
    where TParameter : TechnologicalParameterBaseÐ
    where TContext : DbContextÐ
{Ð
    public async Task<T?> GetCalcMethodAsync(Guid unitId, Guid operationId, Guid parameterId)Ð
    {Ð
        await using TContext context = await ContextFactory.CreateDbContextAsync();Ð
Ð
        return await context.Set<T>()Ð
            .FirstOrDefaultAsync(x => x.UnitId == unitId &&Ð
                x.TechnologicalOperationId == operationId &&Ð
                x.TechnologicalParameterId == parameterId);Ð
    }
}

```

```

    }
}
public async Task<List<T>> GetMethodsByOperationAndUnitAsync(Guid operationId, Guid unitId)
{
    await using TContext context = await ContextFactory.CreateDbContextAsync();

    return await context.Set<T>()
        .Where(x => x.TechnologicalOperationId == operationId && x.UnitId == unitId)
        .Include(x => x.TechnologicalParameter) // Aô>CD3D CCD0CT< D 2D07C =C0KC' ?C @C <CTBD
        .AsNoTracking()
        .ToListAsync();
}
}
</file>

```

```

<file path="Promatis.Net.MES.Service/TechnologicalParameterService/
ITechnologicalParameterService.cs">
using Promatis.Net.MES.Domain;
using Promatis.Net.Service;
namespace Promatis.Net.MES.Service;
public interface ITechnologicalParameterService<T> : IReferenceService<T>
    where T : TechnologicalParameterBase
{
    /// <summary>
    /// Aô>C`CDt8D$L C00D 0CÄ5D$@D²Â >D$DC,,;DÄBD >C$0C0=D`5 C0> D$8C0C CD0C0=D`E (Numeric, String, Boolean, D
    /// </summary>
    Task<List<T>> GetByDataTypeAsync(string dataType);
}
</file>

```

```

<file path="Promatis.Net.MES.Service/TechnologicalParameterService/
TechnologicalParameterService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.MES.Domain;
using Promatis.Net.Service;
namespace Promatis.Net.MES.Service;
public abstract class TechnologicalParameterService<T, TContext>(IDbContextFactory<TContext>
contextFactory)
    : ReferenceService<T, TContext>(contextFactory), ITechnologicalParameterService<T>
    where T : TechnologicalParameterBase
    where TContext : DbContext
{
    public async Task<List<T>> GetByDataTypeAsync(string dataType)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();

        return await context.Set<T>()
            .Where(x => x.DataType == dataType)
            .AsNoTracking()
            .ToListAsync();
    }
}
</file>

```

```

<file path="Promatis.Net.MES.Service/UnitOfMeasurementService/IUnitOfMeasurementService.cs">
using Promatis.Net.MES.Domain;
using Promatis.Net.Service;
namespace Promatis.Net.MES.Service;
/// <summary>
/// A>C0BD 0C0B D 5D 2C,,AC0>C4> D ;Cä0 CD;D0 CC0@C 2C`5C08D0 AC0@C 2CäGC08C0>CÄ 5CD8C08Db 8Ct<CT@CT=C,,0.D
/// </summary>
public interface IUnitOfMeasurementService : IReferenceService<UnitOfMeasurement>
{
}
</file>

```

```

<file path="Promatis.Net.MES.Service/UnitOfMeasurementService/UnitOfMeasurementService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.MES.Data;
using Promatis.Net.MES.Domain;

```

```

using Promatis.Net.Service;
{
namespace Promatis.Net.MES.Service;
{
/// <summary>
/// B 5D 2C„A D4?D 0C$;CT=C„O CÔ>D <C BC„2CÔ>-D ?D 0C$>Dt=Cä9 C„=DD>D <C FC„5C' 5CD8CÔ8Db 8Ct<CT@CT=C„0.
/// </summary>
public class UnitOfMeasurementService(IDbContextFactory<MesApplicationDbContext> contextFactory)
    : ReferenceService<UnitOfMeasurement, MesApplicationDbContext>(contextFactory),
    IUnitOfMeasurementService
{
}
}
</file>

<file path="Promatis.Net.MES.Service/UnitService/IUnitBaseService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;
using Promatis.Net.Service;
{
namespace Promatis.Net.MES.Service;
{
public interface IUnitBaseService<TContext> : IReferenceTreeService<UnitBase, TContext> // <- A„!AÔ A A' AÔ
D$8CÔ0 C" @Cä4C„BCT;DÄAC=8C' 8CÔBCT@DD5C"A!
    where TContext : DbContext
{
    Task<List<UnitBase>> GetByKindAsync(UnitKind kind);
    Task<List<UnitBase>> GetByTypeAsync(UnitType type);
    Task<List<UnitBase>> GetByKindAndTypeAsync(UnitKind kind, UnitType type);
}
}
</file>

<file path="Promatis.Net.MES.Service/UnitService/UnitBaseService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.MES.Domain;
using Promatis.Net.MES.Domain.Interface;
using Promatis.Net.Service;
{
namespace Promatis.Net.MES.Service;
{
public abstract class UnitBaseService<TContext>(IDbContextFactory<TContext> contextFactory)
    : ReferenceTreeService<UnitBase, TContext>(contextFactory), IUnitBaseService<TContext>
    where TContext : DbContext
{
    // =====
    // AÔ A "BD B AT AÔ+A' A !B$ A B$ B' BT#A¢ „ AT A' At#AT"B / AÔ B Aä AÔ Ad )
    // =====
    {
        /// <summary>
        /// A= CÔBD 0C= B DD0C @C„:C,â !C <C @CT0C'8Ct0Dd8Dò 1D44CTB CÔ0Cô8D 0CÔ0 CÔ0 D ;Cä9 CÔ8Cd5
        /// C" :Cä=C= @CTBCÔ>CÄ ACT@C$8D 5, C= D$>D KC' 2C„4C„B CÔ>C'8CÄ>D DCÔKCR :C'0D AD² „FW 'FÖVçEVæ-B 8 CD@.
        /// </summary>
        public abstract override Task<UnitBase> CreateChildTemplateAsync(UnitBase parent);
    }
    // =====
    // Bt B "B' AÄ B$ AD+ A$+A B A, A AÔ+BR „!C$>C"AD$2C ¶-æB 8 Type D$5Cô5D L C$8CD=D² 8CD5C ;DÄ=Cä•
    // =====
    {
        public async Task<List<UnitBase>> GetByKindAsync(UnitKind kind)
        {
            await using TContext context = await ContextFactory.CreateDbContextAsync();
            return await context.Set<UnitBase>().
                .AsNoTracking().
                .Where(x => x.Kind == kind).
                .ToListAsync();
        }
    }
    {
        public async Task<List<UnitBase>> GetByTypeAsync(UnitType type)
        {
            await using TContext context = await ContextFactory.CreateDbContextAsync();
            // A„ACô>C'LCtCCT< CÔ>C 8D$>C$>CR $ ", D$0C¢ :C : UnitType - DÔBCâ ´fÆ w5Ý
            return await context.Set<UnitBase>().
                .AsNoTracking().
                .Where(x => (x.Type & type) != 0).
                .ToListAsync();
        }
    }
}
}

```

```

    }
    public async Task<List<UnitBase>> GetByKindAndTypeAsync(UnitKind kind, UnitType type)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        return await context.Set<UnitBase>()
            .AsNoTracking()
            .Where(x => x.Kind == kind && (x.Type & type) != 0)
            .ToListAsync();
    }
}
</file>

<file path="Promatis.Net.MES.Tests/AssemblyInfo.cs">
// AÄBC;DäGC 5D" ?C @C ;C´5C´LCÔ>CR 2D´?Cä;CÔ5CÔ8CR BCTAD$>C" 2 DÔBCä9 D 1Cä@Cä5D
[assembly: CollectionBehavior(DisableTestParallelization = true)]
</file>

<file path="Promatis.Net.MES.Tests/Promatis.Net.MES.Tests.csproj">
<Project Sdk="Microsoft.NET.Sdk">
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
        <IsPackable>false</IsPackable>
    </PropertyGroup>
    <ItemGroup>
        <PackageReference Include="coverlet.collector" Version="10.0.0">
            <PrivateAssets>all</PrivateAssets>
            <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
        </PackageReference>
        <PackageReference Include="FluentAssertions" Version="8.10.0" />
        <PackageReference Include="FluentValidation" Version="12.1.1" />
        <PackageReference Include="Microsoft.EntityFrameworkCore.InMemory" Version="10.0.8" />
        <PackageReference Include="Microsoft.Extensions.Configuration" Version="10.0.8" />
        <PackageReference Include="Microsoft.NET.Test.Sdk" Version="18.5.1" />
        <PackageReference Include="Moq" Version="4.20.72" />
        <PackageReference Include="xunit.runner.visualstudio" Version="3.1.5">
            <PrivateAssets>all</PrivateAssets>
            <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
        </PackageReference>
        <PackageReference Include="xunit.v3" Version="3.2.2" />
    </ItemGroup>
    <ItemGroup>
        <ProjectReference Include="..\Promatis.Net.Mes.Service\Promatis.Net.MES.Service.csproj" />
    </ItemGroup>
    <ItemGroup>
        <Using Include="Xunit" />
    </ItemGroup>
</Project>
</file>

<file path="Promatis.Net.MES.Tests/Service/UnitServiceStandaloneTests.cs">
using FluentAssertions;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.ChangeTracking;
using Microsoft.Extensions.Configuration;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.MES.Domain.Interface;
using Promatis.Net.MES.Service;
namespace Promatis.Net.MES.Tests;
public class UnitServiceStandaloneTests
{
    // 1. A„ACô>C´LcCCT< C$5Ct4CR FW7DF$6öçFW‡M
    private readonly IDbContextFactory<TestDbContext> _factory;
}

```

```

private readonly DbContextOptions<TestDbContext> _options;
}
public class TestUnit : Domain.UnitBase { }
}
/// <summary>
/// B$5D BCâ2D'9 D 5D 2C,,A CD;Dò 2CT@C,,DC,,:C FC,,8 C 8Ct=CTA-C$KC >D >C¢ Væ-D& 6R 2 CÄ>CDCC'LCÔKDR BCTAD$0
/// A,,!Aô A A' Aô : B 8C4=C BD4@C ?D 8C$5CD5C00 C¢ AD$0C04C @D$C UnitBaseService D >CD=C,,< generic-C00
/// </summary>
public class TestUnitService(IDbContextFactory<TestDbContext> factory)
: UnitBaseService<TestDbContext>(factory)
{
    /// <summary>
    /// Aô@CâAD$5C"HC O Ct0C4;D4HC¤0 DD0C @C,,:C, ?Cä;C,,<Câ@DD8Ct<C 4C'0 D4ACô5D,,=Cä9 C¤>CÄ?C,,;DôFC,,8 C 1
    /// </summary>
    public override Task<Domain.UnitBase> CreateChildTemplateAsync(Domain.UnitBase parent)
    {
        // A,,!Aô A A' Aô : A 5Ct>C00D =Cä5 CD8C00CÄ8Dt5D :Cä5 D >Ct4C =C,,5 C,,=D BC =D 0 C00D$8C$=Cä3Cä B
        var childMock = (Domain.UnitBase)Activator.CreateInstance(parent.GetType());
        childMock.ParentId = parent.Id;
        childMock.Parent = null;
        return Task.FromResult(childMock);
    }
}
public UnitServiceStandaloneTests()
{
    // 2. Aä?Dd8C, 4Cä;Cd=D² 1D'BDÂ 4C'0 TestDbContext
    _options = new DbContextOptionsBuilder<TestDbContext>()
        .UseInMemoryDatabase(Guid.NewGuid().ToString())
        .Options;
    // 3. AÄ>C¤0CT< DD0C @C,,:D2 8CÄ5C0=Cä 4C'0 TestDbContext
    var factoryMock = new Mock<IDbContextFactory<TestDbContext>>();
    factoryMock.Setup(f => f.CreateDbContextAsync(It.IsAny<CancellationToken>()))
        .ReturnsAsync(() => new TestDbContext(_options)); // ReturnsAsync D4?D >D"0CTB Task.FromResult
    _factory = factoryMock.Object;
}
// 4. A¤>C0BCT:D B D$>Cd5 CD>C'6CT= C0@C,,=C,,<C BDÂ ?D 0C$8C'LCÔKCR >C0FC,,8D
public class TestDbContext(DbContextOptions<TestDbContext> options)
: ApplicationDbContext(options, new ConfigurationBuilder().Build())
{
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<Domain.UnitBase>().UseTptMappingStrategy();
        modelBuilder.Entity<TestUnit>().ToTable("TestUnits");
    }
    public override async Task<int> SaveChangesAsync(CancellationToken cancellationToken =
default)
    {
        IEnumerable<EntityEntry<Domain.UnitBase>> entries =
ChangeTracker.Entries<Domain.UnitBase>()
        .Where(e => e.State is EntityState.Added or EntityState.Modified);
        foreach (EntityEntry<Domain.UnitBase> entry in entries)
        {
            Domain.UnitBase unit = entry.Entity;
            if (unit.ParentId.HasValue)
            {
                // At0C4@D46C 5CÂ @Cä4C,,BCT;Dò 4C'0 C0@Cä2CT@C¤8 (C" -äÖVö÷'' f-æB @C 1CäBC 5D" 1D'AD$@Cä
                Domain.UnitBase? parent = Set<Domain.UnitBase>().Find(unit.ParentId.Value);
                if (parent != null)
                {
                    // 1. Aô@C 2C,,;Cä ÷6-F-öâ ,, "CT@CÄ8C00C'LCÔKC' Cct5C²•
                    if (parent.Kind == UnitKind.Position)
                        throw new InvalidOperationException("Position node cannot have
children.");
                }
                // 2. Aô@C 2C,,;Cä FW 'FÖVçB ,, CR <Cä6CTB D >CD5D 6C BDÂ ÷6-F-öâ•
            }
        }
    }
}

```

```

        if (parent.Kind == UnitKind.Department && unit.Kind == UnitKind.Position)
            throw new InvalidOperationException($"A00D CD,,5C08CR 8CT@C @DT8Cf¢ >C JCT:D" :C B
'{parent.Kind}'.");
    }

    // 3. A0@C 2C,,;Câ 8Ct>C´ODd8C, ... &öGV7F-ôâö7F÷& vRöG& ¸7 ÷'B =CR ACÄ5D,,8C$0DäBD O)
    if (parent.Kind is UnitKind.Production or UnitKind.Storage or
UnitKind.Transport)
    {
        if (unit.Kind != parent.Kind && unit.Kind != UnitKind.Position)
            throw new InvalidOperationException($"A00D CD,,5C08CR 8CT@C @DT8Cf¢ w·Væ-Bä||-æ
'{parent.Kind}'.");
    }
}

    }
}

    }

    return await base.SaveChangesAsync(cancellationToken);
}

}

[Fact]
public async Task GetByKindAsync_Should_Return_Units_Matching_Complex_Mask()
{
    // Arrange
    var service = new TestUnitService(_factory);

    // B0GCT9C0 C$ECä4C,,B C" <C AC C Storage
    await service.AddAsync(new TestUnit { Id = Guid.NewGuid(), Name = "Cell_01", Kind =
UnitKind.Storage, Type = UnitType.Cell });
    // B BC =Cä: C$ECä4C,,B C" <C AC C Production
    await service.AddAsync(new TestUnit { Id = Guid.NewGuid(), Name = "Lathe_01", Kind =
UnitKind.Production, Type = UnitType.MachineTool });

    // Act
    List<Domain.UnitBase> storageUnits = await service.GetByKindAsync(UnitKind.Storage);

    // Assert
    storageUnits.Should().ContainSingle(x => x.Name == "Cell_01");
    storageUnits.Should().NotContain(x => x.Name == "Lathe_01");
}

[Fact]
public async Task GetByTypeAsync_Should_Handle_Specific_Flags()
{
    // Arrange
    var service = new TestUnitService(_factory);
    await service.AddAsync(new TestUnit { Id = Guid.NewGuid(), Name = "Crane_1", Kind =
UnitKind.Storage, Type = UnitType.Crane });
    await service.AddAsync(new TestUnit { Id = Guid.NewGuid(), Name = "Vehicle_1", Kind =
UnitKind.Transport, Type = UnitType.Vehicle });

    // Act
    List<Domain.UnitBase> result = await service.GetByTypeAsync(UnitType.Crane);

    // Assert
    result.Should().ContainSingle(x => x.Name == "Crane_1");
}

[Fact]
public async Task MoveAsync_Should_Throw_When_Moving_Into_Position()
{
    // Arrange
    var service = new TestUnitService(_factory);
    var posId = Guid.NewGuid();
    var machineId = Guid.NewGuid();

    await service.AddAsync(new TestUnit { Id = posId, Name = "Terminal Point", Kind =
UnitKind.Position, Type = UnitType.Other });
    await service.AddAsync(new TestUnit { Id = machineId, Name = "Mobile Machine", Kind =
UnitKind.Production, Type = UnitType.MachineTool });

    // Act & Assert
    Func<Task> act = async () => await service.MoveAsync(machineId, posId);
}

```

```

        await act.Should().ThrowAsync<InvalidOperationException>()
            .WithMessage("Position node cannot have children.");
    }

    [Fact]
    public async Task MoveAsync_Should_Prevent_Circular_Dependencies()
    {
        // Arrange
        var service = new TestUnitService(_factory);
        var idA = Guid.NewGuid();
        var idB = Guid.NewGuid();

        // AD>C 0C$;Dô5CÂ ¶-æB 8 Type, D$0C¢ :C : Cä=C, >C 0Ct0D$5C´LC0K (required / init)
        await service.AddAsync(new TestUnit
        {
            Id = idA,
            Name = "Node A",
            Kind = UnitKind.Department,
            Type = UnitType.Workshop
        });

        await service.AddAsync(new TestUnit
        {
            Id = idB,
            Name = "Node B",
            ParentId = idA,
            Kind = UnitKind.Production,
            Type = UnitType.Section
        });

        // Act & Assert
        Func<Task> act = async () => await service.MoveAsync(idA, idB);

        await act.Should().ThrowAsync<InvalidOperationException>()
            .WithMessage("*Bd8C¤;C,,GCTAC¤0Dð 7C 2C,,AC,,<CäAD$L*");
    }

    [Fact]
    public async Task GetFullTreeAsync_Should_Maintain_Hierarchy_And_References()
    {
        // Arrange
        var service = new TestUnitService(_factory);
        var rootId = Guid.NewGuid();
        var sectionId = Guid.NewGuid();

        await service.AddAsync(new TestUnit
        {
            Id = rootId,
            Name = "Dept",
            Kind = UnitKind.Department,
            Type = UnitType.Workshop
        });

        await service.AddAsync(new TestUnit
        {
            Id = sectionId,
            Name = "Section",
            Kind = UnitKind.Production,
            Type = UnitType.Section,
            ParentId = rootId
        });

        await service.AddAsync(new TestUnit
        {
            Id = Guid.NewGuid(),
            Name = "Workstation",
            Kind = UnitKind.Production,
            Type = UnitType.Workstation,
            ParentId = sectionId
        });

        // Act
        Domain.UnitBase? tree = await service.GetFullTreeAsync(rootId);

        // Assert
        tree.Should().NotBeNull();
    }

```



```

        tree!.Children.Should().HaveCount(1);
        tree.Children.First().Children.Should().HaveCount(1);
    }

    [Fact]
    public async Task MoveAsync_Department_Should_Not_Contain_Position()
    {
        // Arrange
        var service = new TestUnitService(_factory);
        var deptId = Guid.NewGuid();
        var positionId = Guid.NewGuid();

        // 1. B >Ct4C 5CÂ CT?C @D$0CÄ5C0B
        await service.AddAsync(new TestUnit
        {
            Id = deptId,
            Name = "Main Dept",
            Kind = UnitKind.Department,
            Type = UnitType.Workshop
        });

        // 2. B >Ct4C 5CÂ Cä7C,,FC,,N (D >C4;C AC0> D$2Cä5C' <C AC¸5, D0BCä <Cä6CTB C KD$L Cell)
        await service.AddAsync(new TestUnit
        {
            Id = positionId,
            Name = "Work Point 1",
            Kind = UnitKind.Position,
            Type = UnitType.Cell
        });

        // Act
        // A0KD$0CT<D 0 C05D 5CÄ5D BC,,BDÂ ÷6-F-öâ 2 Department
        Func<Task> act = async () => await service.MoveAsync(positionId, deptId);

        // Assert
        // Aä6C,,4C 5CÂ >D,,8C :D2 >D" Væ-D& 6T†-W& &6‡•G&-vvW-
        await act.Should().ThrowAsync<InvalidOperationException>()
            .WithMessage($"*C=0D$5C4>D 8C, u ÷6-F-öâr =CR <Cä6CTB C KD$L C$;Cä6CT= C" tFW 'FöVçBrç""½

    }

    [Fact]
    public async Task MoveAsync_Production_Should_Contain_Production_Subnode()
    {
        // Arrange
        var service = new TestUnitService(_factory);
        var parentProdId = Guid.NewGuid();
        var childProdId = Guid.NewGuid();

        await service.AddAsync(new TestUnit
        {
            Id = parentProdId,
            Name = "Production Area",
            Kind = UnitKind.Production,
            Type = UnitType.Workshop
        });

        await service.AddAsync(new TestUnit
        {
            Id = childProdId,
            Name = "Machine Node",
            Kind = UnitKind.Production,
            Type = UnitType.MachineTool
        });

        // Act
        Func<Task> act = async () => await service.MoveAsync(childProdId, parentProdId);

        // Assert
        // At4CTADÂ 8D :C´Ndt5C08D0 1D´BDÂ =CR 4Cä;Cd=CäÂ BC : C¸0Cç &öGV7F-öâ <Cä6CTB D >CD5D 6C BDÂ &öGV7
        await act.Should().NotThrowAsync();

        Domain.UnitBase? updated = await service.GetByIdAsync(childProdId);
        updated!.ParentId.Should().Be(parentProdId);
    }
}
</file>

```

```

<file path="Promatis.Net.MES.Tests/UnitBase/AppDbContext.cs">
using Microsoft.EntityFrameworkCore;
{
namespace Promatis.Net.MES.Tests.UnitBase;
{
// B$5D BCä2D´9 C¤>CÔBCT:D B C$=D4BD 8 Cö@CäAD$@C =>D BC$0 C„<CT= D$5D BCä2D
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }
    // A„ACô>C´LctCCT< C 0Ct>C$KC´ BC„?, DtBCä1D² BD 8C43CT@ CÄ>C2 4CT;C BDÂ 6WCÄVæ–D& 6Sâ,•
    public DbSet<Domain.UnitBase> Units => Set<Domain.UnitBase>();
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // 1. A00D BD 0C„2C 5CÂ E B „:C : C" >D =Cä2CÔ>CÂ ?D >CT:D$5)
        modelBuilder.Entity<Domain.UnitBase>().UseTptMappingStrategy();
        // 2. B4:C 7D´2C 5CÂÂ GD$> TestUnit – DôBCâ =C AC´5CD=C„:
        modelBuilder.Entity<TestUnit>().ToTable("TestUnits");
    }
}
</file>

<file path="Promatis.Net.MES.Tests/UnitBase/TestUnit.cs">
namespace Promatis.Net.MES.Tests.UnitBase;
{
public class TestUnit : MES.Domain.UnitBase { }
</file>

<file path="Promatis.Net.MES.Tests/UnitBase/TestUnitValidator.cs">
using Promatis.Net.MES.Domain;
{
namespace Promatis.Net.MES.Tests.UnitBase;
{
public class TestUnitValidator : UnitBaseValidator<TestUnit> { }
</file>

<file path="Promatis.Net.MES.Tests/UnitBase/UnitBaseArchitectureTests.cs">
using FluentAssertions;
using FluentValidation.TestHelper;
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Data;
using Promatis.Net.MES.Data;
using Promatis.Net.MES.Domain.Interface;
{
namespace Promatis.Net.MES.Tests.UnitBase;
{
public class UnitBaseArchitectureTests
{
    private readonly TestUnitValidator _validator = new();
    private readonly DbContextOptions<AppDbContext> _options;

    public UnitBaseArchitectureTests()
    {
        _options = new DbContextOptionsBuilder<AppDbContext>()
            .UseInMemoryDatabase(Guid.NewGuid().ToString())
            .Options;
    }

    private static EntityChangeEventArgs<MES.Domain.UnitBase> CreateArgs(MES.Domain.UnitBase
entity, DbContext context)
    {
        return new EntityChangeEventArgs<MES.Domain.UnitBase>(
            entity,
            EntityStateChangeEnum.Added,
            new List<PropertyChangeInfo>(),
            context);
    }

    #region A$0C´8CD0Dd8Dò „ C AC¤8 C, 1C 7Cä2D´5 Cö@C 2C„;C •

    [Theory]
    [InlineData(UnitKind.Storage, UnitType.Cell, true)] // BôGCT9C¤0 C" !C¤;C 4CR r C-
    [InlineData(UnitKind.Storage, UnitType.Workshop, false)] // Bd5DR 2 B :C´0CD5 – AâHC„1C¤0 (C 8D" =CR ACä2

```

```

[InlineData(UnitKind.Production, UnitType.Table, true)] // B BCä; C" D >C,,7C$>CDAD$2CR r C-
[InlineData(UnitKind.Position, UnitType.Other, true)] // Aô@CäGCT5 C" Cä7C,,FC,,8 - Aä:ð
[InlineData(UnitKind.Position, UnitType.Table, false)] // B BCä; C" Cä7C,,FC,,8 - AäHC,,1C=0 (D >C4;C ACô>
public void Validator_Should_Check_Bitwise_Compatibility(UnitKind kind, UnitType type, bool
isValid)ð
{ð
    var unit = new TestUnit { Kind = kind, Type = type, Name = "Test" };ð
    TestValidationResult<TestUnit>? result = _validator.TestValidate(unit);ð
ð
    if (isValid)ð
    {ð
        result.ShouldNotHaveAnyValidationErrors();ð
    }ð
    elseð
    {ð
        result.ShouldHaveValidationErrorFor(x => x)ð
            .WithErrorMessage("B$8Cò =CR ACä>D$2CTBD BC$CCTB C=0D$5C4>D 8C,â""½
    }ð
}ð
ð
#endregionð
ð
#region B$@C,,3C45D ,, D 0C$8C´0 C$;Cä6CT=Cô>D BC, ¶-æB•
ð
[Fact]ð
public async Task Trigger_Should_Allow_Root_Units()ð
{ð
    await using var context = new AppDbContext(_options);ð
    var root = new TestUnit { ParentId = null, Kind = UnitKind.Department, Type =
UnitType.Other };ð
ð
    var trigger = new UnitBaseHierarchyTrigger();ð
    EntityCancelEventArgs<Domain.UnitBase> args = CreateArgs(root, context);ð
ð
    await trigger.HandleAsync(args);ð
ð
    args.Cancel.Should().BeFalse();ð
}ð
ð
[Fact]ð
public async Task Trigger_Should_Reject_Position_In_Department()ð
{ð
    await using var context = new AppDbContext(_options);ð
    var parentId = Guid.NewGuid();ð
ð
    context.Units.Add(new TestUnit { Id = parentId, Kind = UnitKind.Department, Type =
UnitType.Other, Name = "Dept" });ð
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);ð
ð
    var child = new TestUnit { ParentId = parentId, Kind = UnitKind.Position, Type =
UnitType.Other, Name = "Pos" };ð
    var trigger = new UnitBaseHierarchyTrigger();ð
    EntityCancelEventArgs<Domain.UnitBase> args = CreateArgs(child, context);ð
ð
    await trigger.HandleAsync(args);ð
ð
    args.Cancel.Should().BeTrue();ð
    args.ErrorMessage.Should().Contain("Cô5 CÄ>Cd5D" 1D´BDÂ 2C´>Cd5Cò 2 'Dept' (Department));ð
}ð
ð
[Theory]ð
[InlineData(UnitKind.Production, UnitKind.Production, false)] // Bd5DR 2 Bd5DR r C-
[InlineData(UnitKind.Production, UnitKind.Transport, true)] // B$@C =D ?Cä@D" 2 Bd5DR r C ?D 5D"5Cô> (
[InlineData(UnitKind.Storage, UnitKind.Position, false)] // Aô>Ct8Dd8Dò 2 B :C´0CB r C-
[InlineData(UnitKind.Position, UnitKind.Position, true)] // Aô>Ct8Dd8Dò 2 Aô>Ct8Dd8Dâ r C ?D 5D"5Cô>
public async Task Trigger_Should_Enforce_Isolation(UnitKind pKind, UnitKind cKind, bool
shouldCancel)ð
{ð
    await using var context = new AppDbContext(_options);ð
    var parentId = Guid.NewGuid();ð
ð
    // B >Ct4C 5CÂ @Cä4C,,BCT;Dò A Cò>CDECä4DôIC,,< D$8Cò>Cí
    context.Units.Add(new TestUnit { Id = parentId, Kind = pKind, Type = GetFirstType(pKind),
Name = "P" });ð
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);ð
ð

```

```

        var child = new TestUnit { ParentId = parentId, Kind = cKind, Type = GetFirstType(cKind),
Name = "C" };
        EntityCancelEventArgs<Domain.UnitBase> args = CreateArgs(child, context);
        await new UnitBaseHierarchyTrigger().HandleAsync(args);
    }
    #endregion
    private static UnitType GetFirstType(UnitKind kind)
    => Enum.GetValues<UnitType>().First(t => t != UnitType.None && ((int)kind & (int)t) != 0);
}
</file>

<file path="Promatis.Net.MES.UI/_Imports.razor">
@using Microsoft.AspNetCore.Components.Forms
@using Microsoft.AspNetCore.Components.Routing
@using Microsoft.AspNetCore.Components.Web
@using MudBlazor
@using Promatis.Net.UI.Components.Workspaces
@using Promatis.Net.UI.Components.Dialogs
</file>

<file path="Promatis.Net.MES.UI/Pages/UnitOfMeasurements/UnitOfMeasurementDialog.razor">
@using Promatis.Net.MES.Domain
@
@* Aä1Cä@C GC,,2C 5CÄ DCä@CÄC C" CCÄ=D´9 C 8Ct=CTA-D,,0C ;Cä= D ?D 0C$>Dt=C,,:Cä2 A0!A,1
Aä= D 0CÄ 2D´7Cä2CTB C 0Ct>C$KC´ :C @C=0D Ä :C0>C0:C, 8 C00C´>Cd8D" ACTBC=C Fluent-C$0C´8CD0Dd8C, □
<ReferenceDialogLayout TModel="UnitOfMeasurement"
    Model="@Model"
    Title="@((Model.Id == Guid.Empty ? "B >Ct4C =C,,5 CT4C,,=C,,FD² 8Ct<CT@CT=C,,0" : "B 5CD0C
    Validator="@Validator" />
</file>

<file path="Promatis.Net.MES.UI/Pages/UnitOfMeasurements/UnitOfMeasurementDialog.razor.cs">
using FluentValidation;
using Microsoft.AspNetCore.Components;
using Promatis.Net.MES.Domain;
@
namespace Promatis.Net.MES.UI.Pages.UnitOfMeasurements;
@
public partial class UnitOfMeasurementDialog : ComponentBase
{
    /// <summary>
    /// A,,=Cd5C=BC,,@D45CÄ AD$@Cä3Cä BC,,?C,,7C,,@Cä2C =C0KC´ fçVVçB02C ;C,,4C BCä@ CD;D0 5CD8C08Db 8Ct<CT@CT=C,,0
    /// Aä= C 2D$>CÄ0D$8Dt5D :C, ?D >C @CäAC,,BD 0 C" DCä@CÄC Dt5D 5Cr ?C @C <CTBD K ReferenceDialogLayout.@
    /// </summary>
    [Inject]
    protected IValidator<UnitOfMeasurement> Validator { get; set; } = null!;
@
    /// <summary>
    /// Ad8C$0D0 @CT4C :D$8D CCT<C 0 CÄ>CD5C´L CD0C0=D´E (Dt8D BC 0 C,,;C, :C´>C0´Ä ?CT@CT4C =C00D0 :Cä=D$5C=PA
    /// A,,<D0 AC$>C"AD$2C $0öFVÄ" AD$@Cä3Cä ACä2C00CD0CTB D ?C @C <CTBD >CÄ 2D´7Cä2C 2 OpenDialogWindowAsy
    /// </summary>
    [Parameter]
    public required UnitOfMeasurement Model { get; set; }
@
    protected override void OnInitialized()
    {
        base.OnInitialized();
        if (Model == null)
            throw new ArgumentNullException(nameof(Model), "A00D 0CÄ5D$@ Model Cä1D07C = C KD$L C05D 5CD0C0 2
    }
}
</file>

<file path="Promatis.Net.MES.UI/Pages/UnitOfMeasurements/UnitOfMeasurementPage.razor">
@page "/mes/mdm/units-of-measurement"
@using Promatis.Net.MES.Domain
@* A00D ;CT4D45CÄ 1C 7Cä2D4N D BD 0C08DdC, C=D$>D 0D0 AC <C @C 7C$5D =CTB D$CC´1C @, D 5D$:D2 8 C 0Ct>C$KCR
@inherits ReferenceGridPage<UnitOfMeasurement>
@
@* A0>D :Cä;DÄ:D2 5CD8C08Dd5 C,,7CÄ5D 5C08D0 4CäAD$0D$>Dt=Cä AD$0C04C @D$=D´E C0>C´5C´ ,, Cä4, A,,<D0Ä C08D 0C0
C00D 0CÄ5D$@ GridColumns CÄK CäAD$0C$;D05CÄ B4!B$+AÄÄ "C 1C´8Dd0 D >C 5D 5D$AD0 AC <C □
<GridColumns></GridColumns>

```

</file>

```
<file path="Promatis.Net.MES.UI/Pages/UnitOfMeasurements/UnitOfMeasurementPage.razor.cs">
using Promatis.Net.MES.Domain;
using Promatis.Net.UI.Components;
using Promatis.Net.UI.Components.Workspaces;

namespace Promatis.Net.MES.UI.Pages.UnitOfMeasurements;

public partial class UnitOfMeasurementPage : ReferenceGridPage<UnitOfMeasurement>
{
    private ReferenceContext<UnitOfMeasurement> _context = null!;

    /// <summary>
    /// B BD >C4> D$8C08Ct8D >C$0C0=D'9 Cä2CT@D 0C"4 D 2Cä9D BC$0 C 0Ct>C$>C' AD$@C =C,,FD²í
    /// A05D 5CD0CTB C4>D$>C$KC' :Cä=D$5C=AD" 2 D04D > ReferenceGridPage CD;D0 @CT=CD5D 8C03C í
    /// </summary>
    protected override ReferenceContext<UnitOfMeasurement> Context => _context;

    protected override void OnInitialized()
    {
        // AÄ AÔ A$ AÔ Aä ACä1C,,@C 5CÄ ?D 8C;C 4C0>C' :Cä<C =CD=D'9 Dd5C0BD 4C'0 CT4C,,=C,,F C,,7CÄ5D 5C08D0í
        // A05D 5CD0CT< D 8D BCT<C0KC' vU6W'f-6U &÷f-FW" 8Cr 1C 7Cä2Cä3Cä :C'0D AC AD$@C =C,,FD²
        // C, ?D 8C$0CtKC$0CT< CÄ5D$>CB <D03C= >C' ?CT@CT7C 3D Cct:C, 3D 8CD0 RefreshGrid.
        _context = new UnitOfMeasurementWorkspaceContext(PageServiceProvider,
onDataChangedNotifier: RefreshGrid);
    }

    // At0C0CD :C 5CÄ 8C08Dd8C ;C,,7C FC,,N C 0Ct>C$KDR ?Cä4C08D >C$ =C ACä1D'BC,,O C" &VfW&Væ6Tw&-E v]
    base.OnInitialized();
}
}
</file>
```

```
<file path="Promatis.Net.MES.UI/Pages/UnitOfMeasurements/UnitOfMeasurementWorkspaceContext.cs">
using MudBlazor;
using Promatis.Net.MES.Domain;
using Promatis.Net.UI.Components;

namespace Promatis.Net.MES.UI.Pages.UnitOfMeasurements;

/// <summary>
/// A0@C,,:C'0CD=Cä9 C= >C0BCT:D B D4?D 0C$;CT=C,,0 D ?D 0C$>Dt=C,,:Cä< CT4C,,=C,,F C,,7CÄ5D 5C08D0 „(D$CC=8, A=8C'>
/// B >CD5D 6C,,B 0 D BD >C$ 8C0DD 0D BD CC=BD4@C0>C' @D4BC,,=D²â $Cä:D4AC,,@D45D$AD0 BCä;DÄ:Cä =C AC$0Ct8 D >
/// </summary>
public class UnitOfMeasurementWorkspaceContext : ReferenceContext<UnitOfMeasurement>
{
    /// <summary>
    /// A= >C0AD$@D4:D$>D ?D 8C08CÄ0CTB D$>C'LC= > CT4C,,=D BC$5C0=D'9 IServiceProvider, C0>C'=CäAD$LDä ;C,,:C$8
    /// A00D 0CÄ5D$@ onDataChangedNotifier (RefreshGrid D BD 0C08DdK) C 2D$>CÄ0D$8Dt5D :C, ?D >C @C AD'2C 5D$A
    /// </summary>
    public UnitOfMeasurementWorkspaceContext(IServiceProvider serviceProvider, Action?
onDataChangedNotifier = null)
    {
        : base(serviceProvider, onDataChangedNotifier: onDataChangedNotifier)
    }
}

/// <summary>
/// B$>Dt:C :Cä=C= @CTBC,,7C FC,,8 C$8CtCC ;DÄ=Cä3Cä >C=C ä
/// B 2D07D'2C 5D" 0C AD$@C :D$=D'5 CRUD-C= >CÄ0C04D² ACä7CD0C08D008Ct<CT=CT=C,,0 D04D 0 D =C BC,,2C0KCÄ vV
/// </summary>
protected override async Task OpenDialogWindowAsync(UnitOfMeasurement model, bool isNew)
{
    // B >Ct4C 5CÄ :Cä=D$5C"=CT@ C00D 0CÄ5D$@Cä2 CD;D0 =C HCT3Cä =C BC,,2C0>C4> Razor-CD8C ;Cä3C DCä@CÄK
    var parameters = new DialogParameters<UnitOfMeasurementDialog>
    {
        { "Model", model } // A05D 5CD0CT< Dt8D BD4N CÄ>CD5C'L C,,;C, 5E 8Ct>C'8D >C$0C0=D'9 C;Cä=D
    };

    var options = new DialogOptions
    {
        CloseButton = true,
        MaxWidth = MaxWidth.Small,
        FullWidth = true
    };

    // A$KctKC$0CT< CD8C ;Cä3 C00C0@D0<D4N C0> CT3Cä =C BC,,2C0>CÄC Razor-D$8C0C0
}
```

```

        IDialogReference dialog = await DialogService.ShowAsync<UnitOfMeasurementDialog>(&
            isNew ? "B >Ct4C =C,,5 CT4C,,=C,,FD² 8Ct<CT@CT=C,,0" : "B 5CD0C=BC,,@Cä2C =C,,5 CT4C,,=C,,FD² 8Ct<CT@CT=C
            parameters,&
            options);&
&
        // Ad4CT< Ct0C=0D´BC,,0 Cä:C00. A @Cä:CT@ D 0CÂ >C =Cä2C,,B Aä B20:D0H DD>D <D² ?D 8 D4AC05D,,=Cä< C= >CÄ
        await dialog.Result;&
    }&
}
</file>

<file path="Promatis.Net.MES.UI/Promatis.Net.MES.UI.csproj">
<Project Sdk="Microsoft.NET.Sdk.Razor">&
&
    <PropertyGroup>&
        <TargetFramework>net10.0</TargetFramework>&
        <Nullable>enable</Nullable>&
        <ImplicitUsings>enable</ImplicitUsings>&
    </PropertyGroup>&
&
    <ItemGroup>&
        <Compile Remove="UnitBase\*" />&
        <Content Remove="UnitBase\*" />&
        <EmbeddedResource Remove="UnitBase\*" />&
        <None Remove="UnitBase\*" />&
    </ItemGroup>&
&
&
    <ItemGroup>&
        <SupportedPlatform Include="browser" />&
    </ItemGroup>&
&
    <ItemGroup>&
        <PackageReference Include="Microsoft.AspNetCore.Components.Web" Version="10.0.8" />&
        <PackageReference Include="MudBlazor" Version="9.4.0" />&
    </ItemGroup>&
&
    <ItemGroup>&
        <ProjectReference Include="...\Core\Configuration.Web\Promatis.Net.Configuration.Web.csproj" /
    >&
        <ProjectReference Include="...\Core\Promatis.Net.UI\Promatis.Net.UI.csproj" />&
        <ProjectReference Include="...\Promatis.Net.MES.Service\Promatis.Net.MES.Service.csproj" />&
    </ItemGroup>&
&
</Project>
</file>

<file path="Promatis.Net.MES.UI/UiModule.cs">
using MudBlazor;&
using Promatis.Net.UI;&
&
namespace Promatis.Net.MES.UI;&
&
public class UiModule : IUiModule&
{&
    public string Name => GetType().Assembly.GetName().Name ?? "Unknown.Module";&
&
    public IEnumerable<(string Title, string Href, string Icon, string? Group)> GetMenuItems()&
    {&
        return new List<(string Title, string Href, string Icon, string? Group)>&
        {&
            // A0;CäAC=8C' AC0@C 2CäGC08C0 BCTEC0>C´>C48Dt5D :C,,E C00D 0CÄ5D$@Cä20
            ("AT4C,,=C,,FD² 8Ct<CT@CT=C,,0", "/mes/mdm/units-of-measurement", Icons.Material.Filled.SettingsInput
        });&
        }&
    }&
}
</file>

<file path="Promatis.Net.MES.UI/wwwroot/exampleJsInterop.js">
// This is a JavaScript module that is loaded on demand. It can export any number of&
// functions, and may import other JavaScript modules if required.&
&
export function showPrompt(message) {&
    return prompt(message, 'Type anything here');&
}
</file>

```

</files>